TDR Cable Fault Detection with Physics-Informed Machine Learning

Finding and classifying coaxial-cable faults from one reflectometry trace: classical baseline vs ML / neural networks, from simulation to real data

[Team members] · Guide: [Guide]
[Institution] · [Course] · November 2026

What this presentation covers

1. Introduction and basics
2. Development journey
3. System architecture
4. Hardware
5. Software
6. Localization (ML/NN)
7. Detection (ML/NN)
8. End-to-end working
9. Hardware–software integration
10. Feature by feature
11. Testing and validation
12. Performance analysis
13. Problems and solutions
14. Achievements and status
15. Limitations and future
16. Conclusion + appendix

The project in one slide

What

Send a voltage step into a cable.

Record the echo with an oscilloscope.

Software answers: is there a fault, which kind, and how far away?

Why

Faults hide inside cables.

Manual reading is slow and subjective.

Weak echoes drown in noise.

Lab instruments add ringing and jitter.

How

Physics simulator makes labelled traces.

Classical baseline vs 5 ML families × 2 inputs.

Calibration ties simulation to the real setup.

Tested on 16 real measured traces (Zenodo).

Status

Software: complete, 62 tests + self-test.

Best distance model: Extra Trees (features).

Best fault-type model: 1-D CNN (raw).

Own lab measurements: pending.

0.37 m

mean distance error, ET-B, every faulted test trace

85 %

of faults the classical baseline locates at all

0.983

macro-F1 of CNN-A over 4 fault classes

19/19

features pass the end-to-end self-test

The real-world problem: faults hidden inside cables

Break (open)

conductor cut → very high impedance → echo positive (Γ = +1)

Short

conductors touching → zero impedance → echo negative (Γ = −1)

Damaged insulation / water

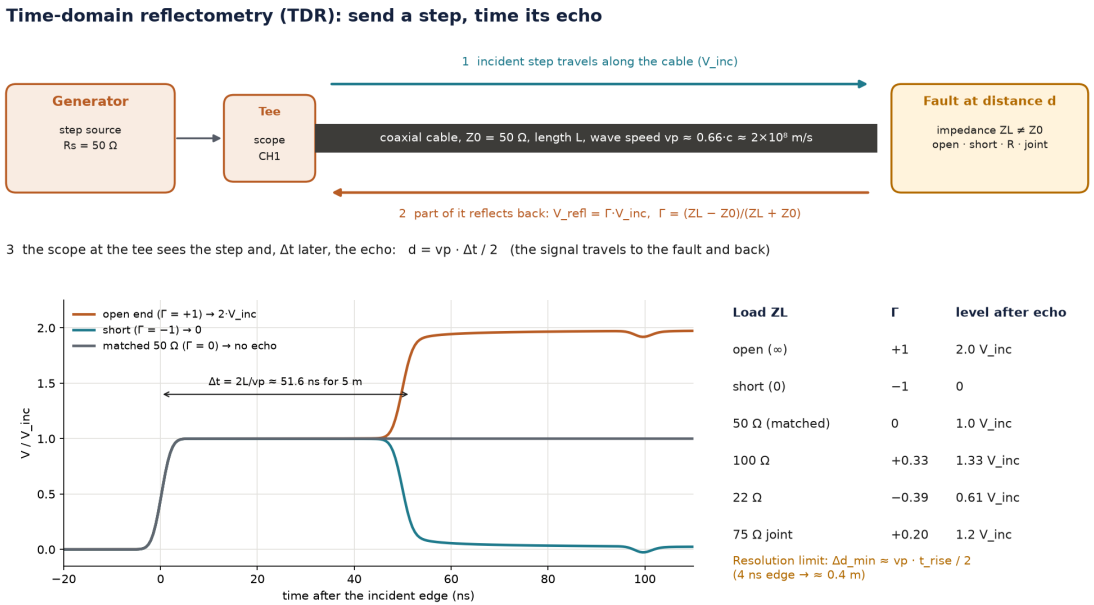
partial leakage path → lower impedance → small negative echo

Bad connector / corrosion

series resistance → higher impedance → small positive echo

Wrong-impedance repair

50 Ω joined to 75 Ω → impedance step → Γ = +0.2



One measurement from one end of the cable can reveal all of these faults.

Why reading the echo is hard

Noise

Small echoes from partial faults sink into noise.

A threshold low enough to see them also fires on noise.

Edge speed (resolution)

Lab generators rise in 5–15 ns.

Faults closer than vp·t_rise/2 (≈ 0.4–1.5 m) merge with the step.

Cable loss and dispersion

The echo returns smaller and slower than it left.

This biases both timing (distance) and height (fault type).

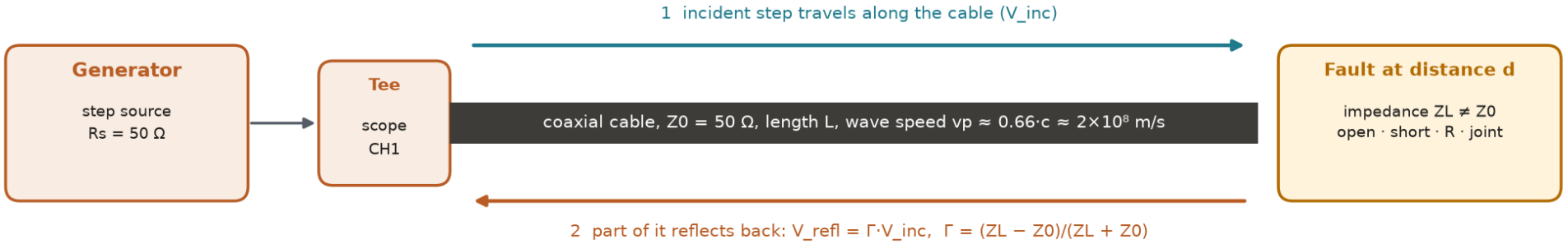
Real instruments

Ringing, adapter reflections, trigger jitter, 8-bit ADC, drift.

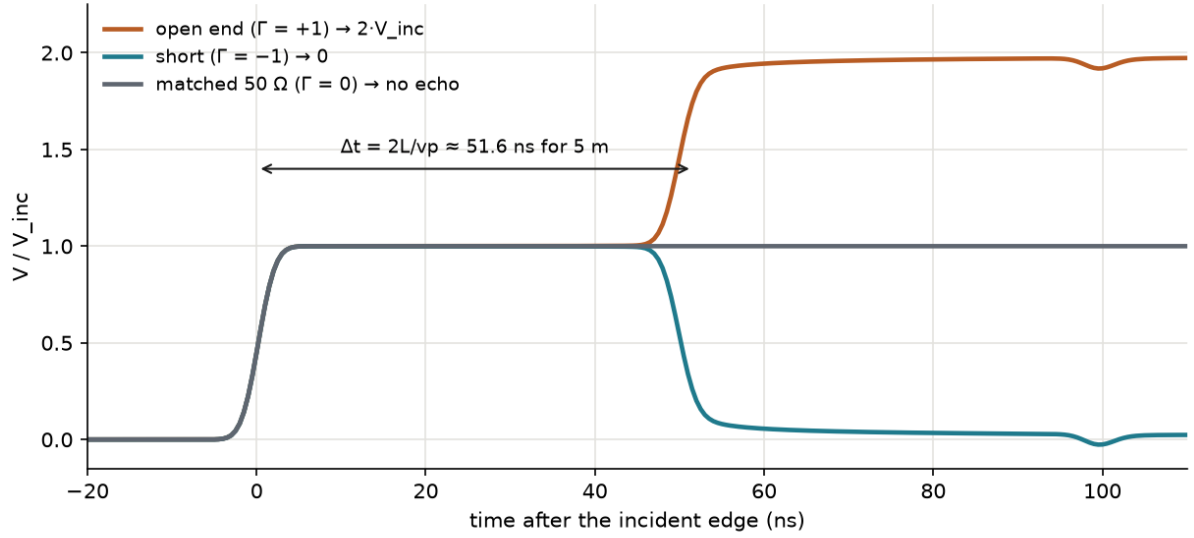
A simple threshold mistakes them for faults.

TDR from zero: send a step, time its echo

Time-domain reflectometry (TDR): send a step, time its echo



3 the scope at the tee sees the step and, Δt later, the echo: $d = vp \cdot \Delta t / 2$ (the signal travels to the fault and back)



Load ZL	Γ	level after echo
open (∞)	+1	2.0 V_inc
short (0)	−1	0
50 Ω (matched)	0	1.0 V_inc
100 Ω	+0.33	1.33 V_inc
22 Ω	−0.39	0.61 V_inc
75 Ω joint	+0.20	1.2 V_inc

Resolution limit: $\Delta d_{min} \approx vp \cdot t_{rise} / 2$
(4 ns edge → ≈ 0.4 m)

Key terms used in this presentation

Term	Meaning	Term	Meaning
TDR	time-domain reflectometry: send a step, time its echo	SNR	signal-to-noise ratio, 20·log10(V_inc/σ) in dB
Z0	characteristic impedance of the cable (50 Ω for RG-58)	Rise time t_r	10–90 % time of the edge; sets the resolution
Γ (gamma)	reflection coefficient (ZL − Z0)/(ZL + Z0)	ML / NN	machine learning / neural network
V_inc	height of the incident step at the tee	RF / ET	Random Forest / Extra Trees: averages of many decision trees
vp, VF	wave speed in the cable; VF = vp/c ≈ 0.66	k-NN	k-nearest neighbours: answer of the most similar training traces
Δt	round-trip time of the echo; d = vp·Δt/2	MLP / CNN	multi-layer perceptron / convolutional neural network
Localization	estimating the distance d to the fault	Method A / B	model input: raw trace (256 samples) / 23 physics features
Detection	deciding fault vs no fault and its class	MAE / F1	mean absolute error (m) / harmonic mean of precision and recall

Existing methods and their limitations

Method	How it works	Limitation
Manual scope reading	operator measures Δt with cursors	slow, subjective, impossible for small echoes in noise
Threshold / peak detection	first derivative lobe above a noise threshold	misses weak echoes at low SNR; ringing causes false early detections
Commercial TDR instruments	dedicated fast-edge hardware + vendor software	expensive, closed algorithms, not in a student lab
Matched filter (Shin et al. 2005)	correlate with the expected echo shape	needs a known pulse shape; still a threshold at the end
Published ML on TDR (Scarpetta 2021, Slimani 2024/25, Smail 2011)	CNN / k-NN / neural inverse model	each uses its own data; rarely compared with a classical baseline on identical traces

Our angle: five model families × two inputs vs a classical baseline on identical traces, then the same chain on real data.

Our proposed solution

1 Physics first

transmission-line simulator → unlimited labelled traces

2 One processing chain

simulated and measured traces go through identical code

3 Fair comparison

5 families × 2 inputs vs baseline, chosen on validation only

4 Calibration

vp, t0, rise, V_inc measured → simulator re-centred

5 Real-data test

16 measured traces + 1 M simulated (Zenodo 5720438)

6 Lab tooling

auto-capture, labelled log, live check, beginner hardware guide

Objectives and expected outcome

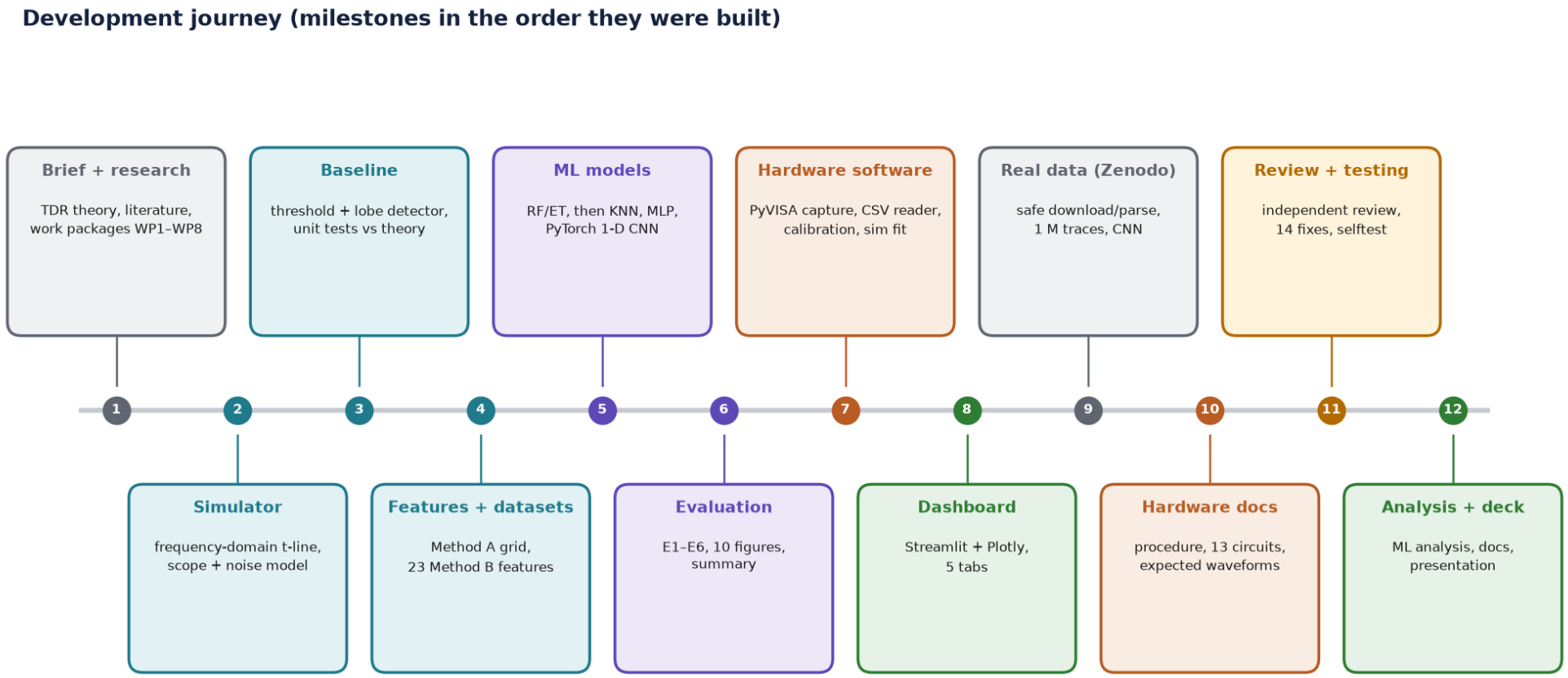
#	Objective	Implemented in	Status
O1	Simulate realistic labelled TDR traces	src/sim_tdr.py	Done, physics-checked
O2	Classical baseline to beat	src/baseline.py	Done
O3	ML for distance + fault type; raw vs features	features.py, models.py, train.py	Done, 18 models
O4	Experiments E1–E6	src/evaluate.py, analysis.py	Done on simulation
O5	Lab setup, calibration, labelled real data	capture.py, calibrate.py, hardware/	Software done; lab pending
O6	Interactive demonstrator	app/dashboard.py	Done
O7	Validate on real measured traces	src/zenodo.py	Done (16 Zenodo traces)

Part 2

Development journey

How the project went from a brief to a tested system, stage by stage

Milestones in the order they were built



Started 2026-11-05 from the project brief; software, simulation and real-data results complete; lab measurements (hardware steps S1–S14) pending lab access.

Research and planning: from literature to model choice

Source	What it contributed	What we built
Brief (WP1–WP8)	RF / ET with physics features, E1–E6	baseline, RF-B, ET-B, experiments
Smail et al. 2011	train a neural network on physics simulations, invert on real data	simulator + MLP
Slimani et al. 2024/25	k-NN beat a 1-D CNN for localisation (RG-58 simulations)	k-NN on both inputs
Scarpetta et al. 2021	1-D CNN on pulse TDR + public dataset with real traces	1-D CNN, Zenodo pipeline
Shin et al. 2005	cross-correlation detection, resolution limited by rise time	matched-filter baseline (Zenodo)
Chang & Park 2019	time-frequency CNN	noted as future work (P3-7)

Requirements we set before coding

Functional

Locate a fault (distance in m) from one trace.

Classify: open / short / matched / partial.

Work with any common lab scope (unknown model).

Calibrate the setup from measurements.

Compare classical vs ML on identical traces.

Show results interactively.

Non-functional

Runs on a student laptop (CPU only).

Reproducible: fixed seeds, stored settings.

Fair: hyper-parameters chosen on validation only.

Safe with downloaded data (no code execution).

Usable without hardware (simulated scope).

Every feature testable automatically.

Technology selection and why

Need	Choice	Why this one
Numerics, signal processing	NumPy, SciPy	FFT-based simulator, filters, least-squares fit
Classical ML	scikit-learn	RF, ET, k-NN, MLP with one fit/predict interface
Neural network (1-D CNN)	PyTorch (CPU)	flexible, runs on a laptop, streams large datasets
Instrument control	PyVISA + pyvisa-py	one API for USB/LAN scopes; driver-free fallback
Dashboard	Streamlit + Plotly	interactive web app from one Python file, zoomable plots
Figures	Matplotlib, schemdraw	report figures; circuit diagrams as code
Data	pandas, joblib	tables, model bundles
Testing	pytest + own self-test	unit tests + end-to-end runs in a sandbox

Development environment and first run

```
# 1. Python environment (Windows, Python 3.11+)
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
.venv\Scripts\pip install torch --index-url https://download.pytorch.org/whl/cpu

# 2. Check packages, VISA backends, calibration, models
.venv\Scripts\python index.py check

# 3. Train and evaluate on simulation (about 1 h on a laptop)
.venv\Scripts\python index.py train
.venv\Scripts\python index.py evaluate
.venv\Scripts\python index.py analyze

# 4. Prove every feature works (temporary folder, no hardware)
.venv\Scripts\python index.py selftest

# 5. Demo
.venv\Scripts\python index.py dashboard
```

What each step gives

requirements.txt pins numpy, scipy, pandas, matplotlib, scikit-learn, streamlit, plotly, pyvisa, pytest, schemdraw.

check: [ok] per package, VISA backend, calibration and model status.

train: models/tdr_models.joblib (18 models).

evaluate / analyze: report/results + figures.

selftest: report/results/feature_tests.md.

Stage by stage: what, why, input, output

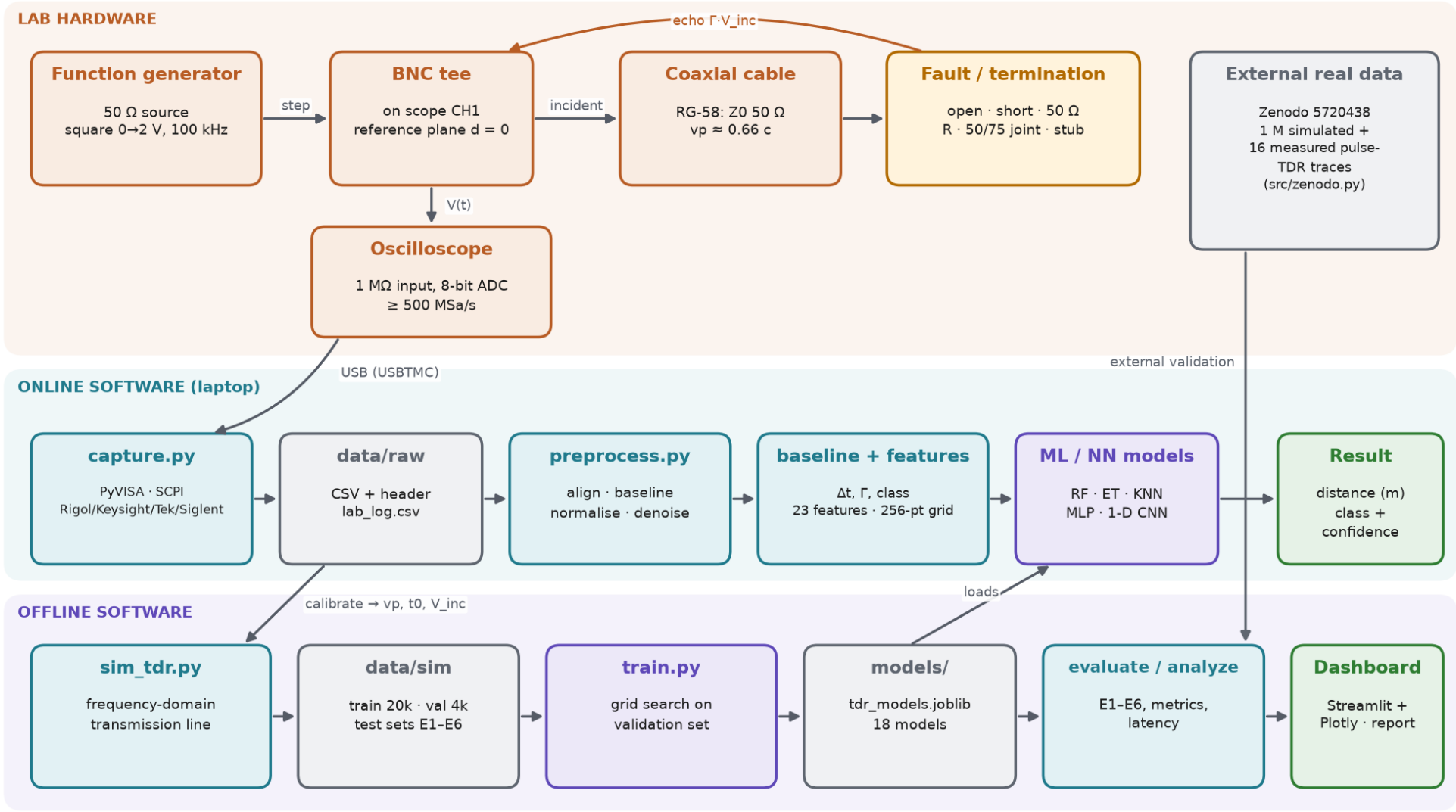
Stage	What we did / why	Input → output	Challenge → resolution
Simulator	frequency-domain transmission line, all fault kinds: labelled data at no cost	parameters → (t, v) + labels	tail of incident edge looked like an echo → shared lobe boundary
Baseline	threshold + lobe detector: the reference to beat	trace → Δt, d, Γ, class	noise false alarms → MAD noise, adaptive smoothing
Features / datasets	Method A grid, 23 features; cached sets	traces → X (n×256), F (n×23)	stale caches → config fingerprint in names
Models + training	5 families, grid search on validation	train/val → bundle	no sample weights in k-NN/MLP → row repetition
Evaluation	E1–E6, 10 figures, then deep analysis	bundle + test sets → tables	unfair comparison → 'fair' subset metric
Lab software	PyVISA capture, universal CSV, calibration, fit	scope → CSV + log → calibration.json	unknown scope → 4 vendor drivers + CSV fallback
Real data	Zenodo 5720438, 4.18 GB, safe parsing	archive → Z1–Z3 results	pickled labels → restricted unpickler
Review + tests	independent review: 14 findings, 12 fixed, 1 documented, 1 not occurring; self-test	code → fixes + evidence	see Part 13

Part 3

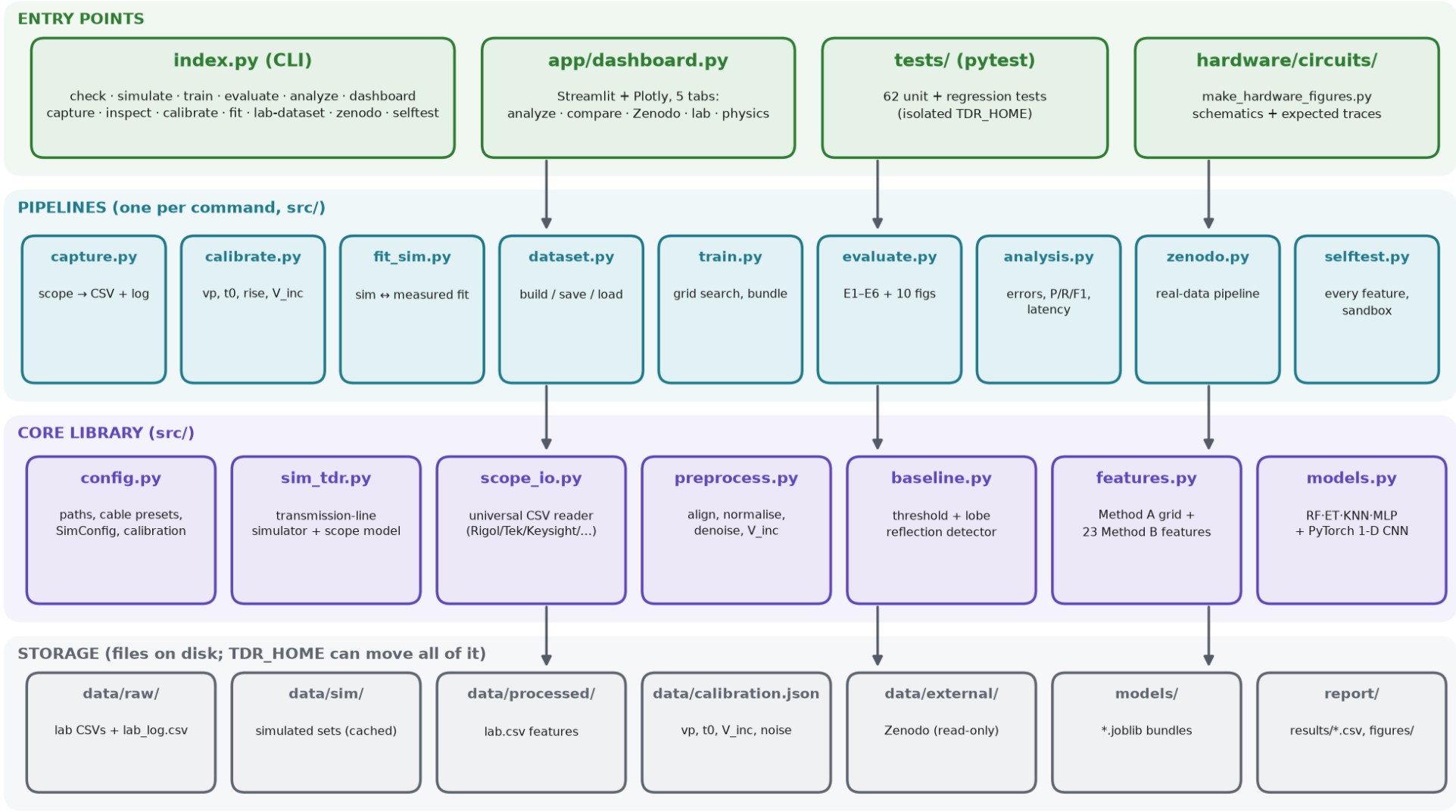
System architecture

Three layers: lab hardware, online software per trace, offline simulation/training/evaluation

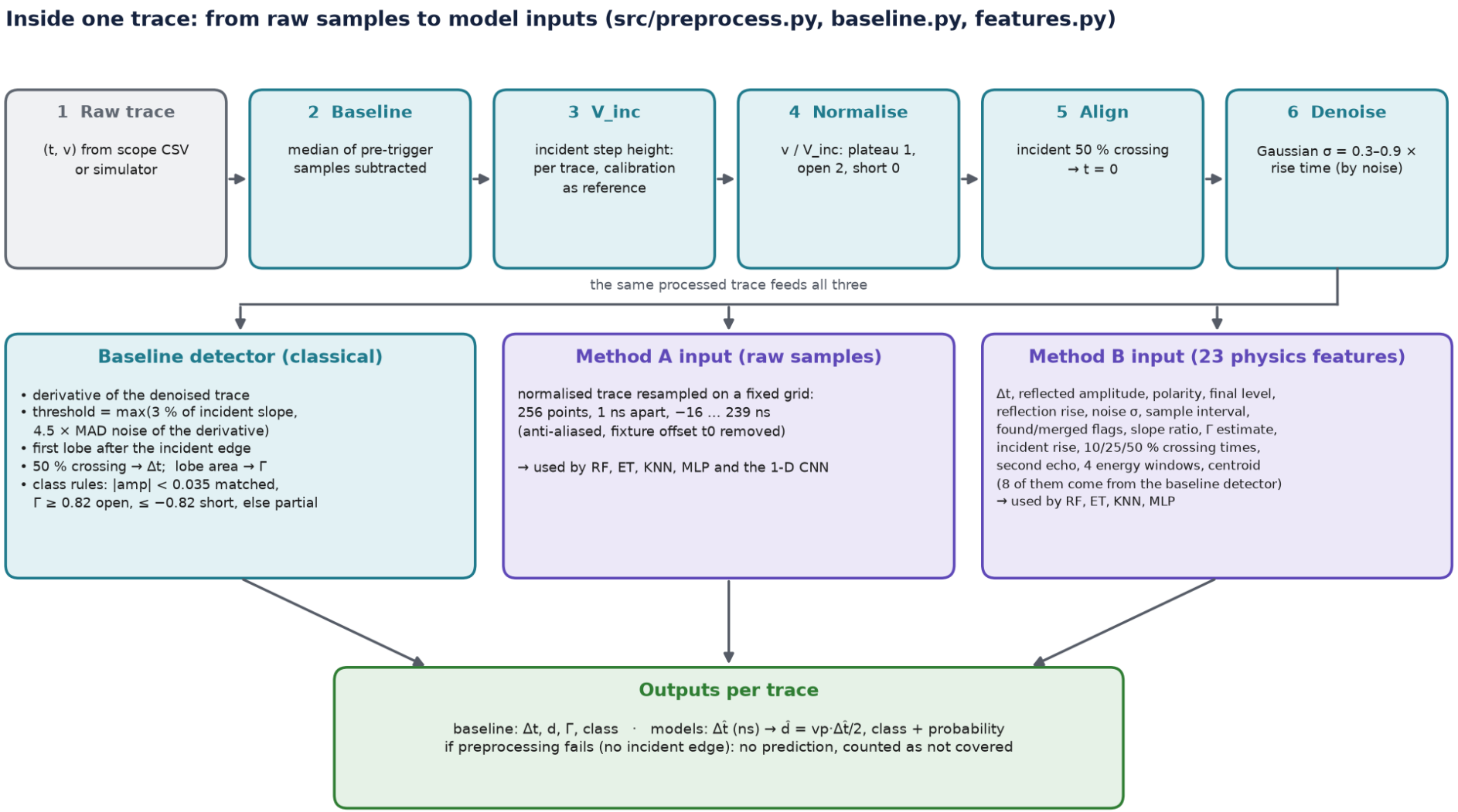
Overall system block diagram



Software architecture: four layers

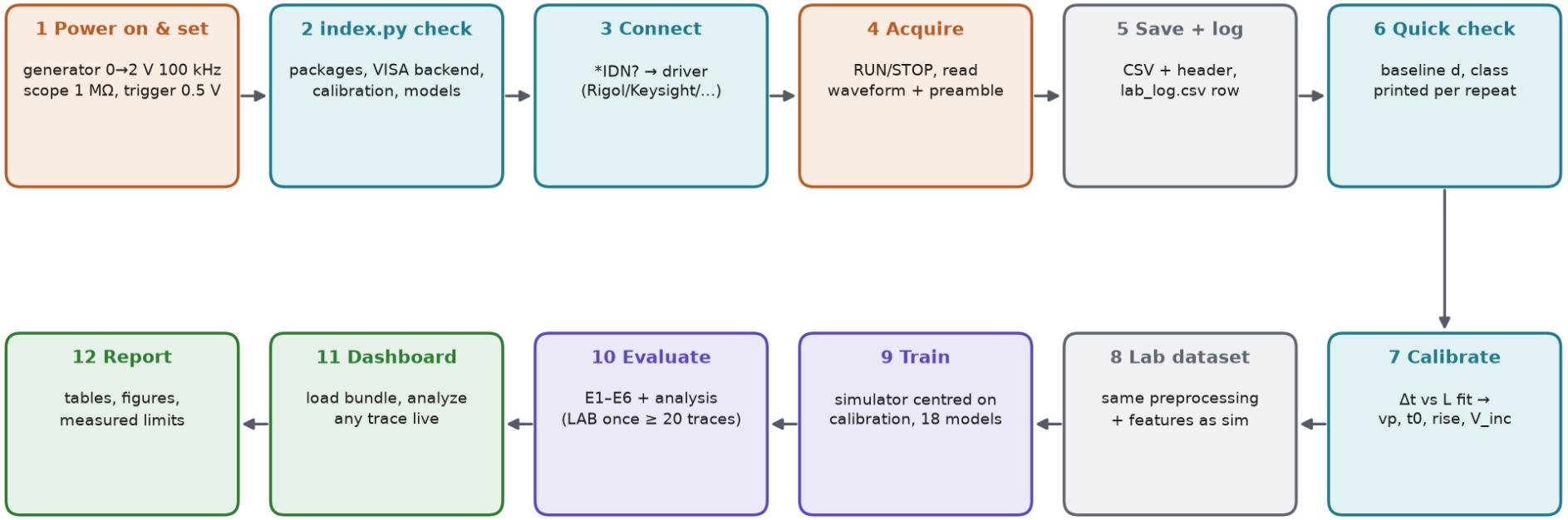


Data flow inside one trace



Complete workflow, start to finish

End-to-end run: from power-on to a located fault



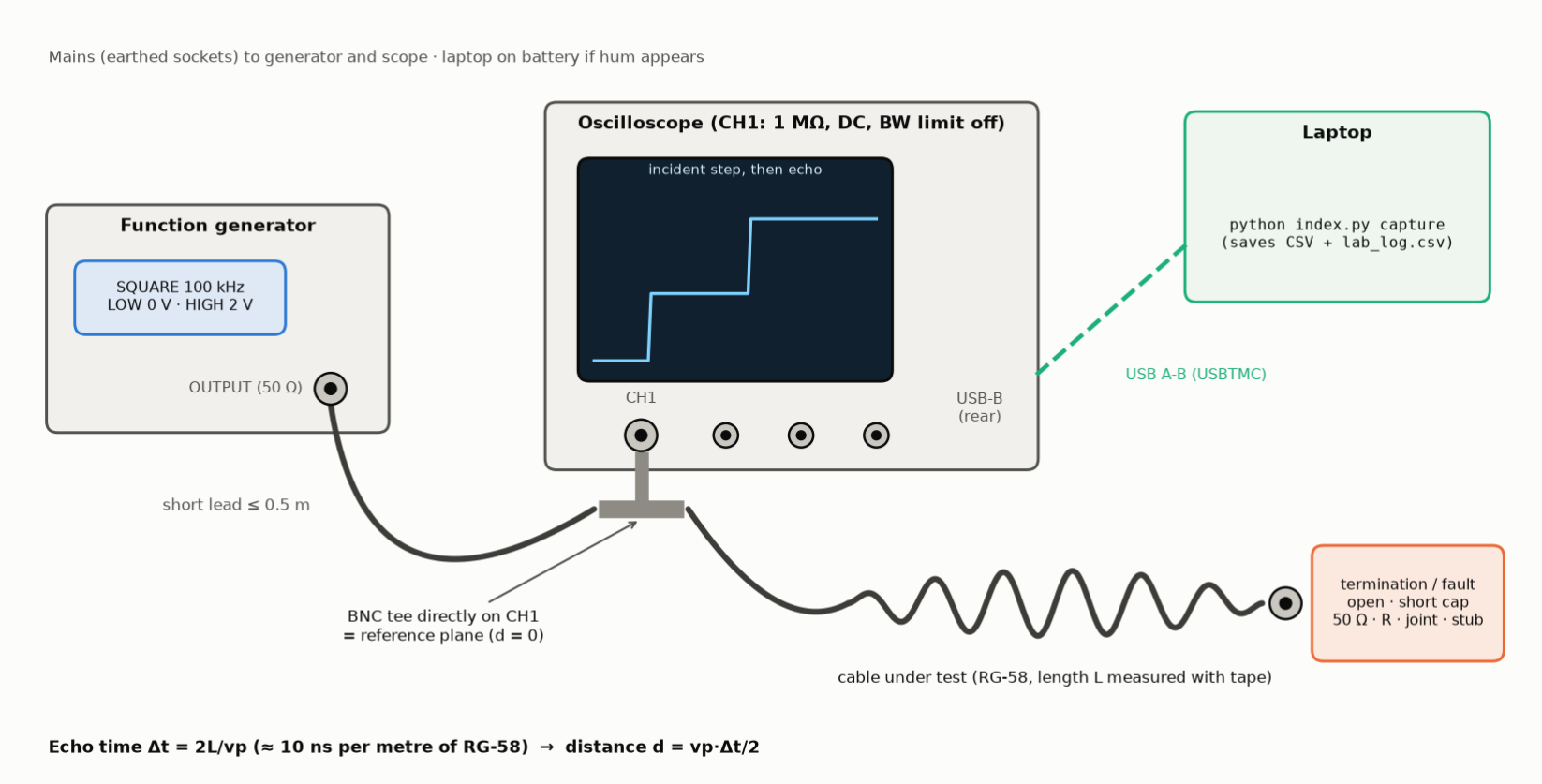
Steps 1–6 repeat for every configuration (H5–H8); 7–12 run once after the session. Steps 2–12 are tested end to end with the simulated scope (index.py selftest).

Part 4

Hardware

Standard lab instruments, coaxial cables and a few passive parts. No microcontroller, no firmware. Built in the order of hardware/README.md

Hardware set-up at a glance



- **Generator:** 50 Ω source, square 0 → 2 V, 100 kHz
- **Tee on CH1:** joins generator, cable and scope
- **Cable:** RG-58, 2 / 5 / 10 m
- **Load:** open, short, 50 Ω or an approved fault
- **Laptop:** USB-TMC to the scope, runs capture

Component inventory (bill of materials)

Component	Specification	Purpose / why selected
Function generator	50 Ω out; 0 V/2 V levels; edge ≤ 15 ns (≤ 5 ns better)	makes the incident step; 50 Ω absorbs returning echoes
Digital oscilloscope	≥ 100 MHz, ≥ 500 MSa/s, 1 MΩ, USBTMC, CSV export	records the step and echo; bandwidth sets rise time
Coaxial cables RG-58	Z0 50 Ω, VF ≈ 0.66; 2, 5, 10 m (+ 3, 2, 1 m pieces)	lines under test; ≥ 2 lengths needed for calibration
BNC tee ×2, short lead ≤ 0.5 m	50 Ω, UG-274 style	reference plane at CH1; second tee for shunt/stub
Terminations	50 Ω end + feed-through, shorting cap, 75 Ω	matched / short / joint loads
Fault parts	22, 49.9, 100 Ω 1 % 0.25 W; binding-post adapters; series box; RG-59	partial faults with known Γ
Tools	multimeter, tape (mm), USB stick, camera	check parts, lengths, CSV fallback, setup photo
Optional pulser	74AC14, 4 × 180 Ω, 5 V	≈ 2–3 ns edge for better resolution

Component deep-dive: generator, oscilloscope, cable

Function generator

Principle: internal source Vs behind Rs = 50 Ω.

Use: square 0→2 V, 100 kHz; high time 5 µs ≫ round trip.

I/O: OUTPUT BNC → short lead → tee.

Power: mains, earthed socket.

Expect: 2.0 V open, 1.0 V into 50 Ω (H3).

Problem: plateau doubled → 'Load = 50 Ω' display setting only.

Oscilloscope

Principle: samples V(t) with an 8-bit ADC after a trigger.

Use: CH1 1 MΩ, DC, BW limit off, trigger rising 0.5 V Normal.

I/O: CH1 BNC in; USB-B (USBTMC) or LAN out.

Expect: step to 1 V, echo at 2L/vp.

Problem: half-height plateau → input set to 50 Ω.

Coax + tee + loads

Principle: wave travels at vp; impedance changes reflect.

RG-58: Z0 50 Ω, VF 0.66, 16 dB/100 m @100 MHz.

Tee: male stem on CH1, two female ports.

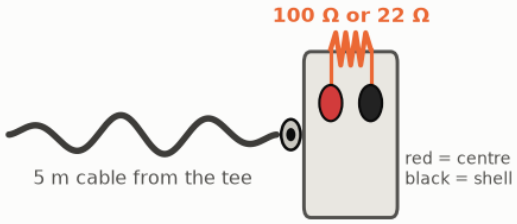
Loads: open (Γ +1), short (−1), 50 Ω (0).

Problem: extra echoes → long lead or tee not on CH1.

Fault fixtures: how each discontinuity is assembled

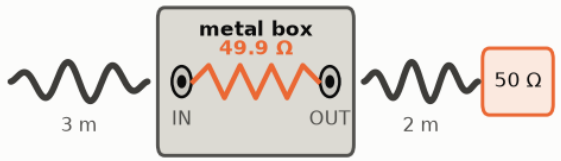
Fault fixtures: how each approved discontinuity is assembled (all start at tee side port 2)

H7a resistor at the cable end



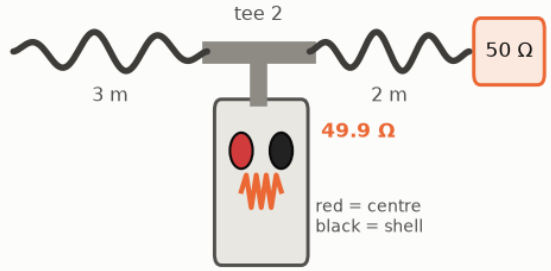
BNC → binding-post adapter; resistor leads ≤ 5 mm.
Measure R with the multimeter, log the measured value (--zl).

H7b series-resistor box (build, §8.2)



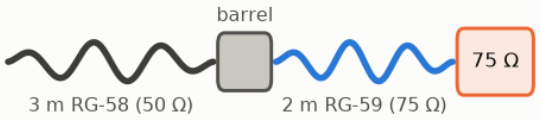
Resistor between the two centre pins; the box joins the shells.
Check: centre-centre ≈ 49.9 Ω, shell-shell < 0.5 Ω.

H7c shunt resistor on a 2nd tee



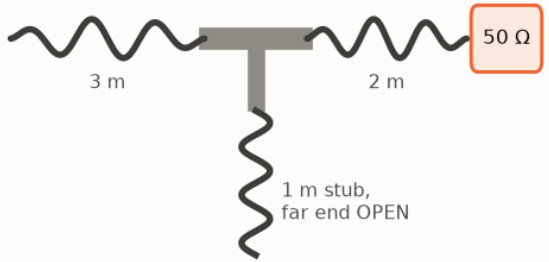
Stem → binding-post adapter + resistor.
End of the line matched, so only the shunt reflects.

H7d 50/75 Ω joint



Γ = (75 − 50)/(75 + 50) = +0.2 at the barrel; the 75 Ω end is matched.

H7e open stub on a 2nd tee



Dip to ≈ 0.67 V lasting 2·l_stub/vp (≈ 10 ns for 1 m).

End loads (S8, S9)

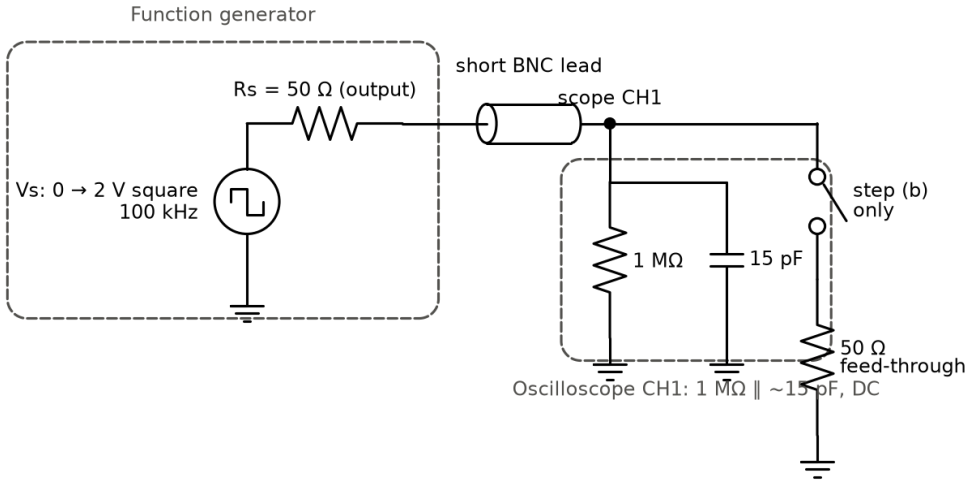


open
nothing on the end (Γ = +1 → 2 V)

short cap
centre tied to shell (Γ = −1 → 0 V)

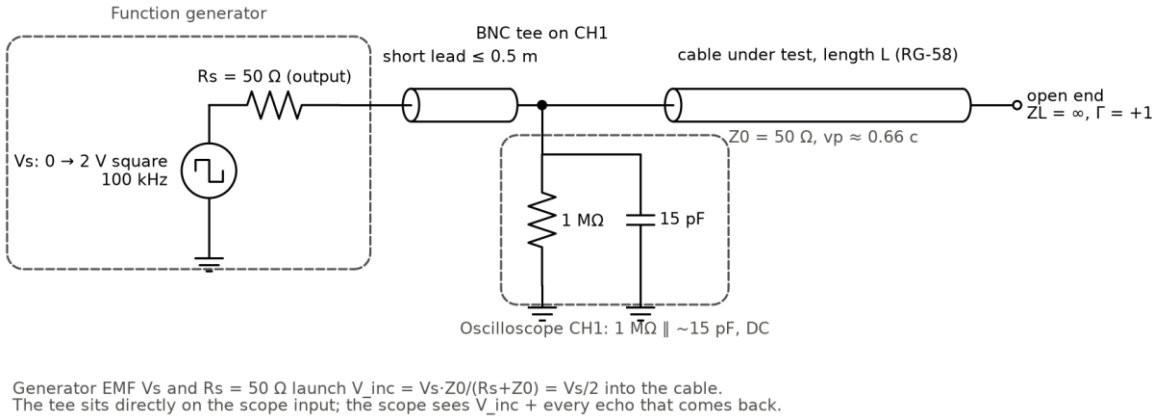
50 Ω terminator
matched (Γ = 0 → flat 1 V)

Circuit diagrams: generator check and basic TDR set-up



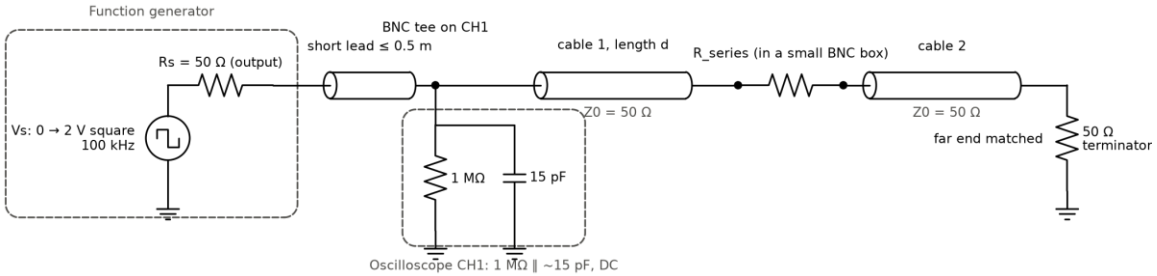
(a) no terminator: read V_open (1 MΩ ≈ open)
Rs = 50 Ω × (V_open / V_50 − 1)

(b) 50 Ω feed-through on CH1: read V_50 and the 10-90 % rise time

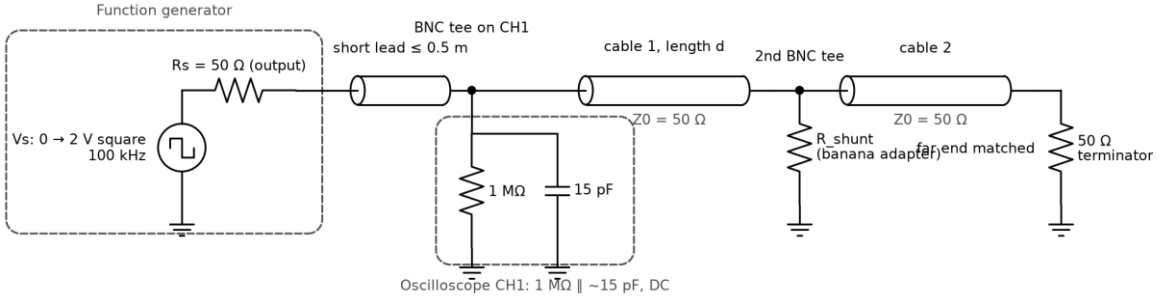


Generator EMF Vs and Rs = 50 Ω launch V_inc = Vs·Z0/(Rs+Z0) = Vs/2 into the cable.
The tee sits directly on the scope input; the scope sees V_inc + every echo that comes back.

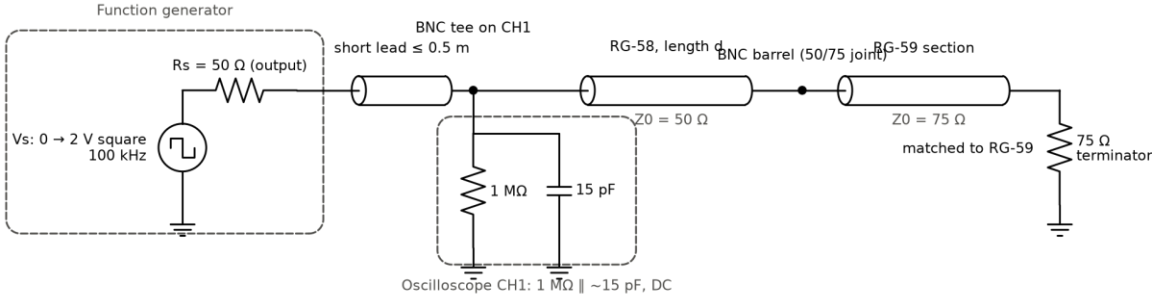
Circuit diagrams of the partial faults



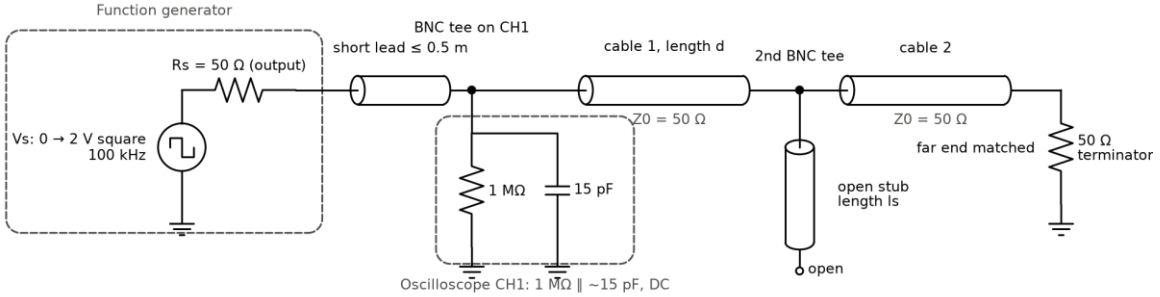
Series resistor at distance d: Γ = R/(R + 2·Z0), e.g. R = 50 Ω → Γ = +0.33 (step UP at 2d/vp).



Shunt resistor at distance d (partial short): Γ = −Z0/(Z0 + 2R), e.g. R = 50 Ω → Γ = −0.33 (step DOWN).

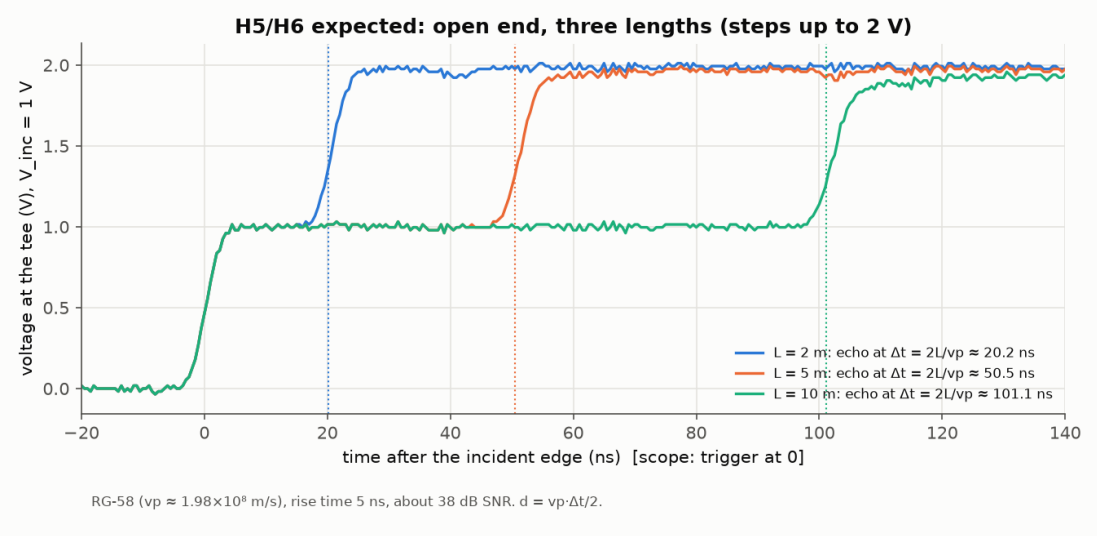


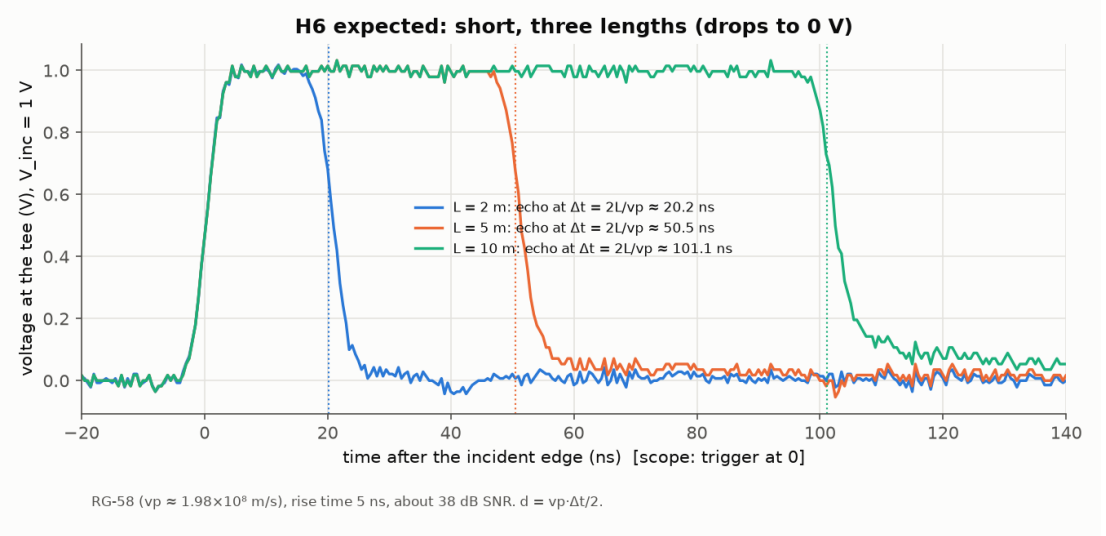
Impedance step at distance d: Γ = (75 − 50)/(75 + 50) = +0.20 (step UP at 2d/vp).

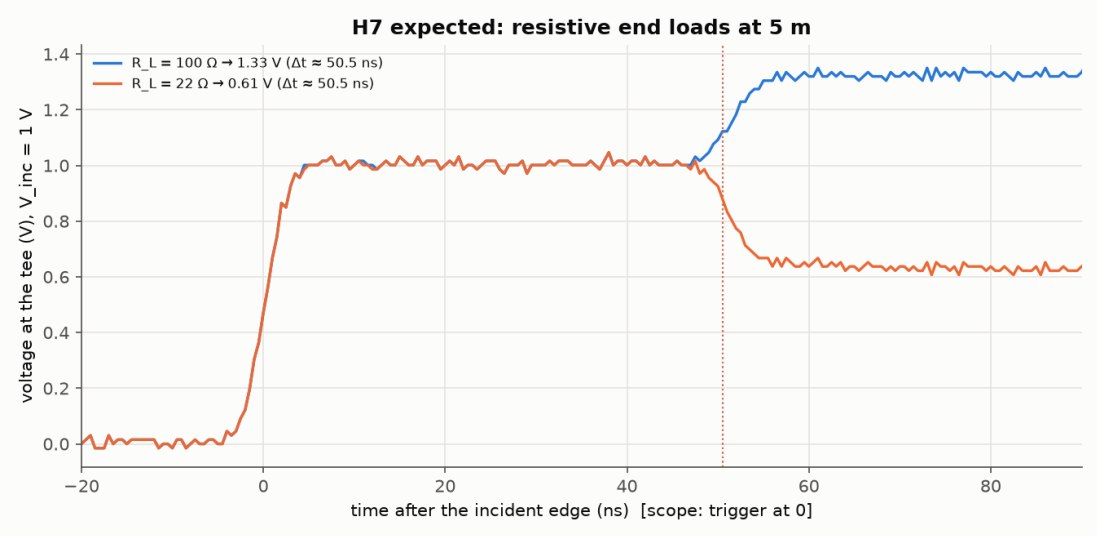


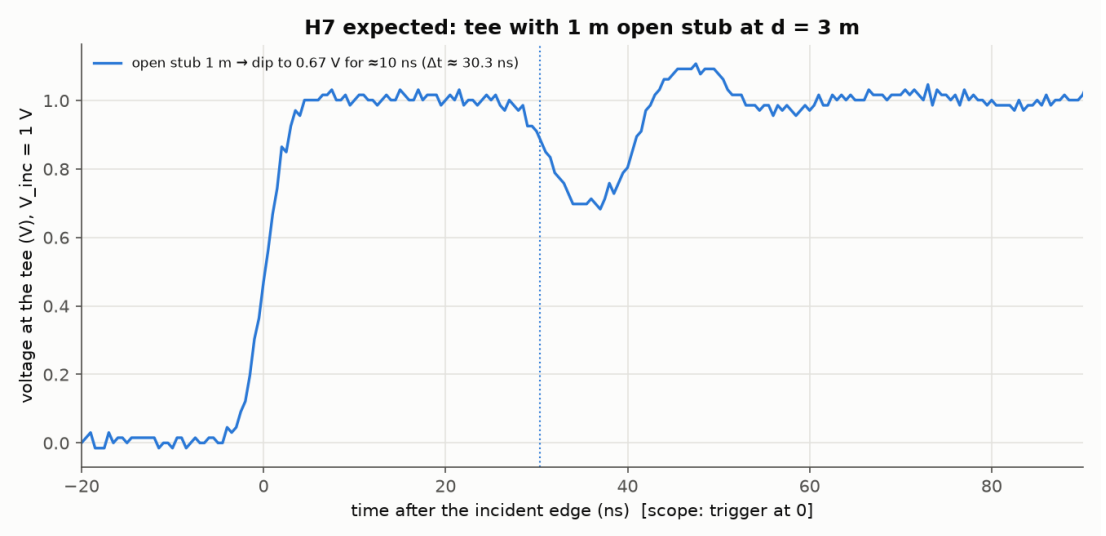
Tee + open stub at distance d: dip of −V_inc/3 that lasts 2·ls/vp, then the level returns (Γ = −1/3).

Expected scope pictures (simulated before the lab)







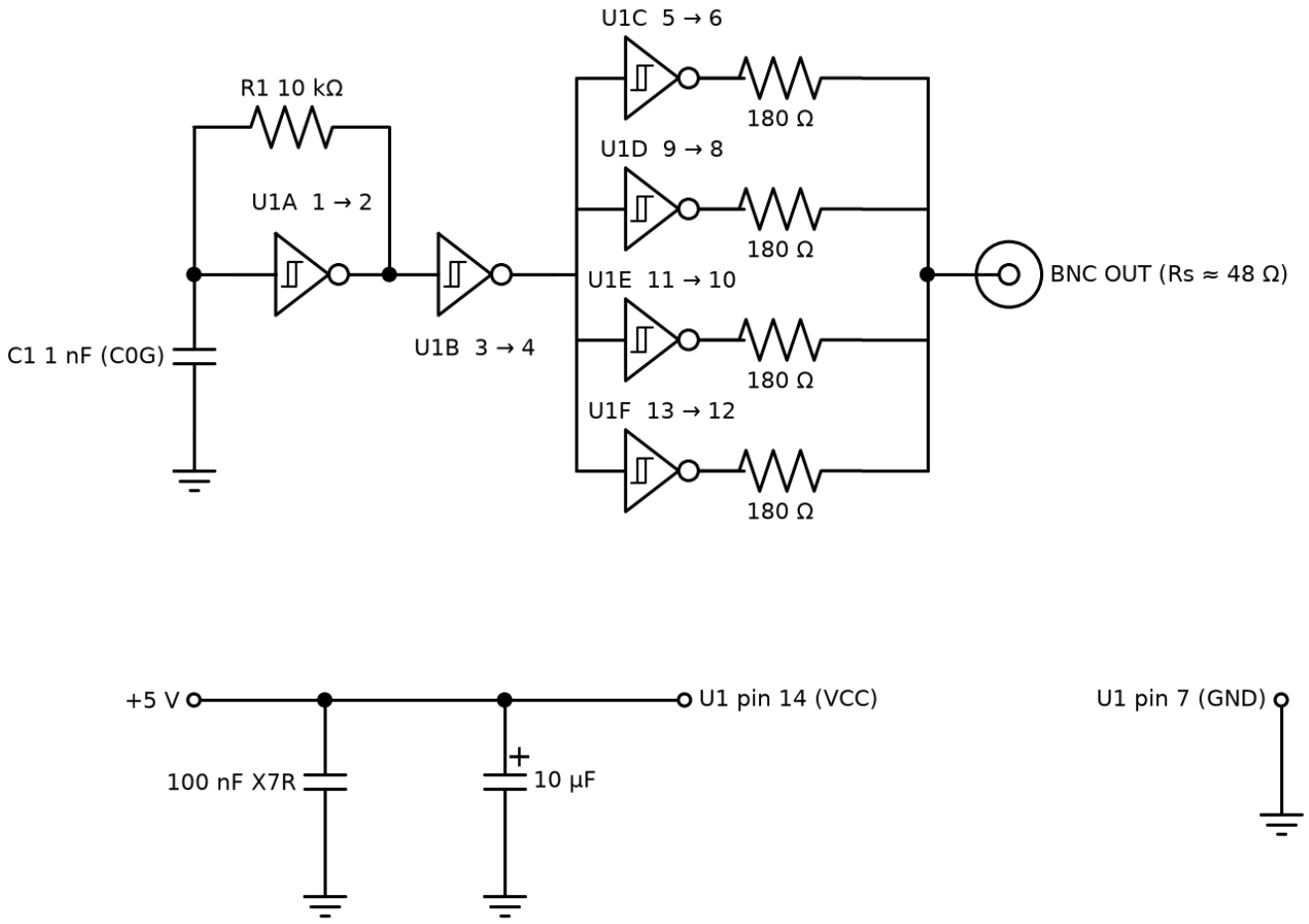


Port-to-port connection table (set-up B)

#	From (port, connector)	Via	To (port, connector)
B1	Scope CH1 input, BNC female	–	tee male stem, directly on CH1
B2	Generator OUTPUT, BNC female	short lead ≤ 0.5 m (M–M)	tee side port 1
B3	Tee side port 2	–	cable under test, end A
B4	Cable end B	–	open / shorting cap / 50 Ω terminator / fault
B5	Laptop USB-A	USB A-B cable	scope rear USB Device (USBTMC)

Ground reference: BNC shells, tied to mains earth through both instruments' earthed sockets.

Optional fast-edge pulser: schematic and pinout



Pin	Connects to
1 (1A)	R1 10 kΩ + C1 1 nF
2 (1Y)	R1 (feedback), pin 3
4 (2Y)	pins 5, 9, 11, 13
6, 8, 10, 12	180 Ω each → BNC centre
7 GND	ground plane, BNC shell
14 VCC	+5 V, 100 nF + 10 µF

74AC14 (AC family, not HC). Decoupling capacitors directly at pins 14/7. Output path < 1 cm. Only with instructor approval.

Power distribution and safety

Equipment	Supply	Draw / note
Generator, scope	mains via earthed sockets	< 100 W each; never defeat the earth pin
Laptop	battery or charger	run on battery if 50/60 Hz hum appears
Cables, loads	none (passive)	max ≈ 17 mW in a 22 Ω load
Pulser (optional)	5.0 V ± 0.25 V, ≥ 150 mA	≈ 10 mA idle, ≈ 105 mA peak into a short

Safety rules

Generator ≤ 5 V peak (2 V used); an open end doubles it.

Scope on 1 MΩ; never the 50 Ω input above its rating.

Discharge long cables before connecting.

Generator output OFF while changing loads.

Only instructor-approved modifications.

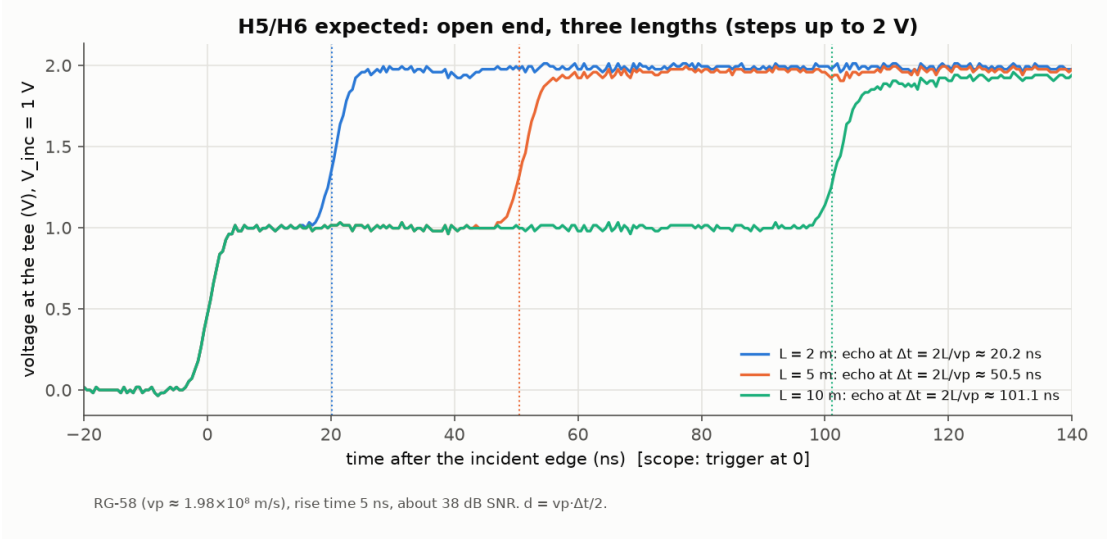
No high-voltage (avalanche) pulsers.

Hardware assembly and test sequence (S1–S15)

Step	Task	Done when	Status
S1–S2	approvals; collect and multimeter-check parts	every BOM row ticked, parts verified	pending (lab)
S3	laptop install + rehearsal	check ok, selftest no Fail	ready
S4–S5	equipment discovery; laptop ↔ scope	*IDN? string printed	pending (lab)
S6–S7	generator check (Rs 45–55 Ω); basic set-up	flat 1 V plateau, echo where expected	pending (lab)
S8	calibration: open at ≥ 2 lengths + matched	VF 0.64–0.68, σ(vp)/vp < 1 %, |t0| < 3 ns	pending (lab)
S9–S11	test matrix, partial faults, low-SNR	≈ 180 + fault + low-SNR traces logged	pending (lab)
S12–S14	data check + backup; process; report	E3/E4 marked LAB	software ready
S15	optional pulser	Rs 45–55 Ω, faster edge	optional

Calibration: method and rehearsal result

- **Measure:** 20 open-circuit traces at each of ≥ 2 lengths (2, 5, 10 m), 10 matched traces.
- **Fit:** straight line Δt vs L → slope 2/vp, intercept t0 (tee + trigger delay).
- **Also:** rise time → resolution vp·t_r/2; V_inc and noise at the normal amplitude.
- **Used by:** every later command automatically (calibration.json).
- **Rehearsal (simulated scope):** vp = 1.9616e+08 m/s vs simulated truth 1.9786e+08 (-0.86 %); t_offset 0.23 ns; rise 4.15 ns; resolution 0.41 m; SNR 30.5 dB



Status: method tested; real calibration values pending the lab session.

Part 5

Software

Python 3.13 · NumPy/SciPy · scikit-learn · PyTorch · PyVISA · Streamlit/Plotly · pytest

Folder structure

```
D:\EMT
├─ index.py                one CLI for every step
├─ src/
│   ├─ config.py            paths, cable presets, SimConfig
│   ├─ sim_tdr.py           transmission-line simulator
│   ├─ preprocess.py        align · normalise · denoise
│   ├─ baseline.py          classical detector
│   ├─ features.py          Method A grid, 23 features
│   ├─ dataset.py           build / save / load sets
│   ├─ models.py            RF·ET·KNN·MLP·1-D CNN
│   ├─ train.py · evaluate.py · analysis.py
│   ├─ scope_io.py · capture.py · calibrate.py · fit_sim.py
│   ├─ zenodo.py            external real data
│   └─ selftest.py          every feature end to end
├─ app/dashboard.py       Streamlit + Plotly
├─ tests/                 62 pytest tests
├─ hardware/              manual, procedure, circuits/
├─ docs/  report/         documentation, results, figures
└─ presentation/          this deck + diagrams
```

Main execution flow

1 index.py parses the command.

2 Dispatch to src/<module>.main() inside a CLI error handler.

3 config.load_calibration() merges calibration.json with defaults.

4 The module reads its inputs (traces, datasets, bundle).

5 Writes outputs to data/, models/ or report/.

6 Wrong inputs: one-line error, exit code ≠ 0 (TDR_DEBUG=1 for a traceback).

Modules: purpose, inputs and outputs (1/2)

Module	Purpose	Input → output	Key algorithm
config.py	paths, presets, ranges, calibration	calibration.json → settings	merge with datasheet defaults; cfg fingerprint
sim_tdr.py	make labelled traces	params → (t, v), labels	Γ_in(f) recursion, IFFT, scope model
scope_io.py	read any scope CSV	file → Waveform	dialect detection, unit parsing
preprocess.py	normalise every trace	(t, v) → Processed	MAD noise, V_inc, 50 % alignment, Gaussian
baseline.py	classical detection	Processed → Δt, d, Γ, class	derivative threshold + lobe walk
features.py	model inputs	Processed → grid, 23 features	resampling, crossings, energies
models.py	model zoo	params → estimator	sklearn pipelines, PyTorch CNN

Modules: purpose, inputs and outputs (2/2)

Module	Purpose	Input → output
dataset.py	build/save/load simulated and lab sets	traces + labels → CSV, X.npy, raw.npz
train.py	grid search on validation, bundle	train/val → models/tdr_models.joblib
evaluate.py	E1–E6, ten figures, summary	bundle + test sets → report/results
analysis.py	error stats, P/R/F1, latency, sizes	bundle + test sets → analysis_*.csv, a1–a6
capture.py	scope capture, labelled log, quick check	VISA scope → CSV + lab_log.csv
calibrate.py / fit_sim.py	vp, t0, rise, V_inc; simulator fit	open traces → calibration.json, fig0 / fig5
zenodo.py	external dataset pipeline	4.18 GB archive → Z1–Z3
selftest.py	run every feature in a sandbox	→ feature_tests.md
app/dashboard.py	interactive analysis	bundle + results + traces → web UI

Inside the simulator (sim_tdr.py)

```
# per frequency f (simplified from transfer_function)
gamma = alpha(f) + 1j*2*pi*f/vp          # cable: loss + delay
G_d   = fault_reflection(kind, ZL, R, Z2, stub, f)
G_in  = G_d * exp(-2*gamma*d)             # back to the input
Z_in  = Z0 * (1 + G_in) / (1 - G_in)
Z_tee = Z_in || Z_scope(1 MOhm, C)         # scope at the tee
H     = Z_tee / (Rs + Z_tee)               # divider with generator
H    *= gaussian_edge(f, rise)
v = cumsum(irfft(H)) * Vs                  # step response
v = scope(v, jitter, drift, noise@SNR, 8-bit)
```

- **Fault kinds:** end load, series R, shunt R, impedance step, open stub.
- **Loss:** $\alpha(f) = \alpha_{dc} + k_s\sqrt{f} + k_d \cdot f$ from datasheet points.
- **Randomised:** d, Γ, rise 2–10 ns, SNR 10–40 dB, Rs ±8 %, scope C, sampling.
- **Checked:** final level = 1 + Γ within 0.015, delay = 2L/vp (F09).

Inside preprocessing: one normalised trace from any source

```
def preprocess_measured(t, v, cal, v_inc=None):
    est = preprocess(t, v, v_inc=None)     # estimate from this trace
    ref = cal['v_inc_v'] if calibrated else None
    use_ref = (not est.ok                  # estimate failed
               or 0.8 <= est.v_inc/ref <= 1.25   # same amplitude
               or est.rise_inc > 1.6*cal['rise_time_s'])  # echo merged
    return preprocess(t, v, v_inc=ref) if use_ref else est
```

- **1 Baseline:** median of pre-trigger samples removed.
- **2 V_inc:** plateau after the first big rise.
- **3 Normalise:** plateau = 1, open → 2, short → 0.
- **4 Align:** incident 50 % crossing → t = 0 (removes trigger jitter).
- **5 Denoise:** Gaussian, 0.3–0.9 × rise time depending on noise.
- **6 Grid:** 256 points, 1 ns, −16…239 ns (Method A).

Same code for simulated, Zenodo-style and lab traces: the models never see a different preprocessing.

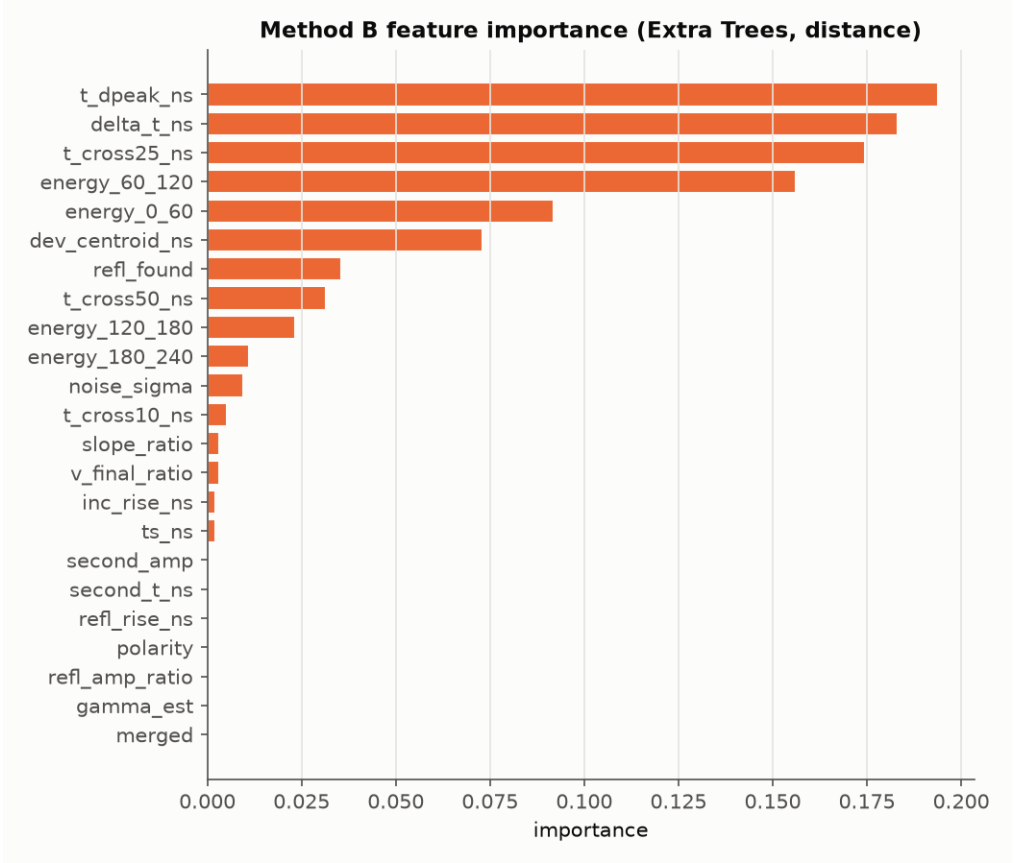
Inside the classical baseline (baseline.py)

```
ds  = d/dt smooth(trace)
thr = max(0.03*peak_incident_slope, 4.5*MAD(ds))
for lobe in lobes_after_incident(ds, thr):
    if abs(lobe.area) >= 0.02:   # skip ADC / noise lobes
        dt = t50(lobe) - t_offset; amp = lobe.area; break
d     = vp*dt/2
gamma = amp*exp(2*alpha(0.1/rise)*d)
cls   = matched if |amp|<0.035 else open if gamma>=0.82
        else short if gamma<=-0.82 else partial
```

- **Output:** found?, Δt, d, amplitude, Γ, class, second echo.
- **Merged mode:** same-polarity echo inside the incident edge → Δt from the lobe centroid.
- **Strength:** very accurate when it finds the echo.
- **Weakness:** misses weak echoes at low SNR; ringing → false early lobes.

Method B: the 23 physics features

Group	Features
Timing	delta_t_ns, t_dpeak_ns, t_cross10/25/50_ns, second_t_ns, dev_centroid_ns
Amplitude / sign	refl_amp_ratio, polarity, v_final_ratio, gamma_est, second_amp, slope_ratio
Shape	refl_rise_ns, inc_rise_ns, energy_0_60, energy_60_120, energy_120_180, energy_180_240
Quality / flags	noise_sigma, ts_ns, refl_found, merged



Instrument control (capture.py)

```
class Rigol(ScopeDriver):
    def read(self, ch):
        self.w(f':WAV:SOUR CHAN{ch}')
        self.w(':WAV:MODE NORM'); self.w(':WAV:FORM BYTE')
        pre = [float(x) for x in self.q(':WAV:PRE?').split(',')]
        raw = self.inst.query_binary_values(':WAV:DATA?', datatype='B')
        v = (raw - yorig - yref) * yinc     # from the preamble
        t = (arange(n) - xref) * xinc + xorig
        return t, v
```

Vendor	Command set
Rigol	:WAV:PRE? / :WAV:DATA?
Keysight/Agilent	:WAVeform:PREamble? / DATA?
Tektronix	WFMOutpre? / CURVE?
Siglent (X-E)	C1:WF? DAT2 + TRDL?
unknown	Keysight-style fallback
--simulate	SimScope (no hardware)

Every repeat: acquire → save CSV with all labels → append lab_log.csv → print a live baseline check.

Dashboard: analyze any trace live

Deploy

TDR Fault Locator

Not calibrated yet: datasheet values in use

Cable

RG-58

Propagation velocity vp (m/s)

1.9786e+8

Velocity factor 0.660

Trace source

- Simulate
- Upload scope CSV
- Saved trace

Simulated setup

Fault

Open end

Fault distance d (m)

5.00

SNR (dB)

30

System rise time (ns)

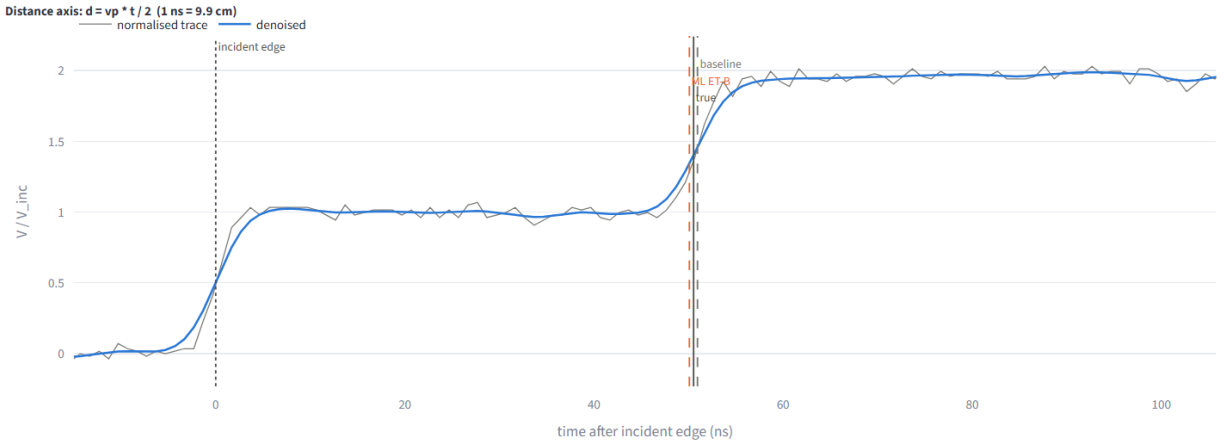
4.00

Scope sample interval (ns)

1.0

Analyze trace | Model comparison | Real data (Zenodo) | Calibration & lab | Physics

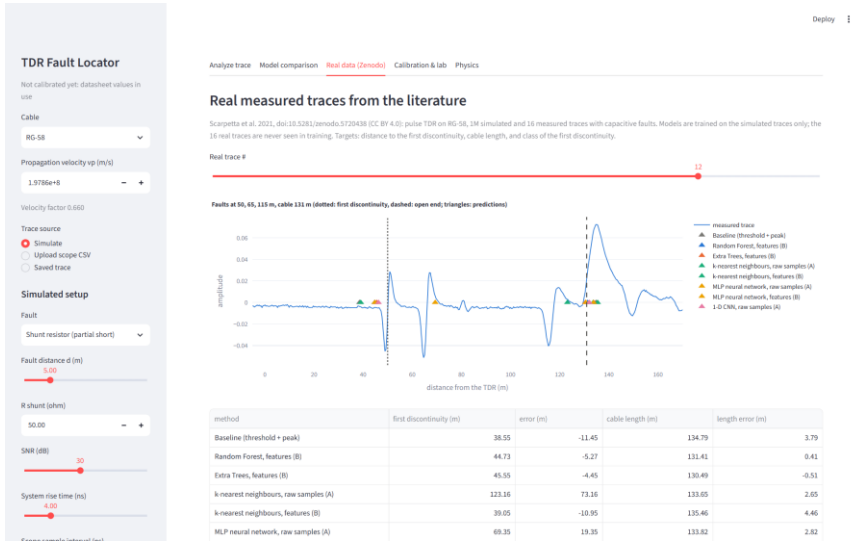
Baseline distance	ML distance (ET-B)	ML class (CNN-A)	Baseline class / Gamma	Ground truth
5.04 m	4.96 m	open	open	5.00 m
↑ +0.04 m vs true	↓ -0.04 m vs true	↑ 100% confidence	↑ Gamma ~ +0.97	↑ open, Gamma +1.00

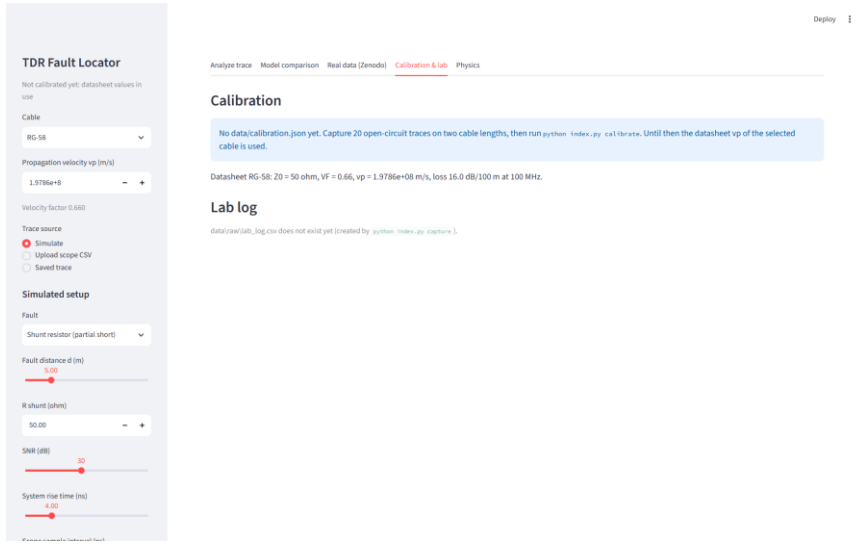


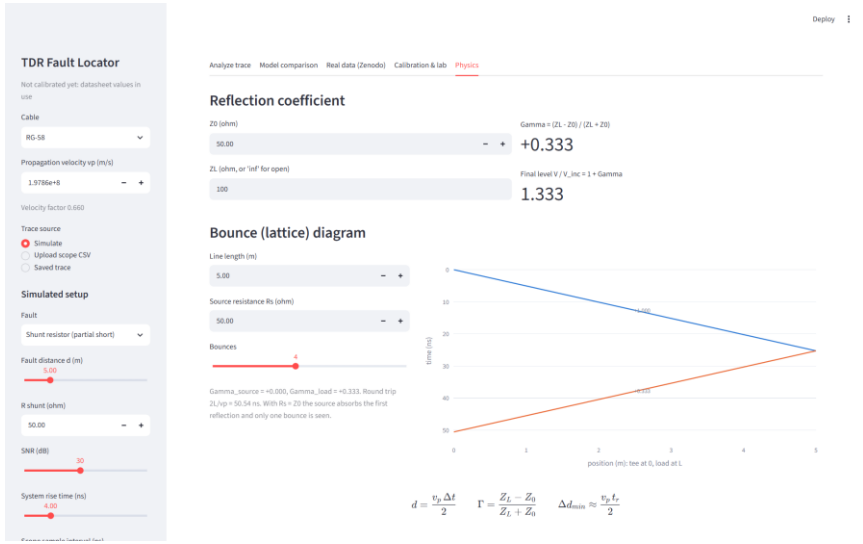
method	dt (ns)	distance (m)	class	confidence	error (m)
Baseline (threshold + peak)	50.953	5.041	open	None	0.041
Random Forest, raw samples (A)	50.613	5.007	open	1.000	0.007
Random Forest, features (B)	50.185	4.965	open	1.000	-0.035
Extra Trees, raw samples (A)	50.387	4.985	open	1.000	-0.015
Extra Trees, features (B)	50.097	4.956	open	1.000	-0.044
k-nearest neighbours, raw samples (A)	50.127	4.959	open	1.000	-0.041
k-nearest neighbours, features (B)	50.439	4.990	open	1.000	-0.010

Dashboard: the other four tabs









Error handling, isolation and data safety

Error handling

CLI-wide handler: missing file, bad CSV, bad value → one-line 'error: …', exit 1.

argparse errors → usage + exit 2.

Dashboard: unreadable file → sidebar message, app keeps running.

Failed preprocessing → no prediction (not a made-up one).

TDR_DEBUG=1 shows tracebacks.

Isolation and reproducibility

TDR_HOME moves data/, models/, report/.

pytest and selftest run in temporary folders.

Seeds fixed per dataset; config fingerprint in cache names.

Bundle stores selected params, forest size, training-set name, sim config.

Downloaded data safety

Zenodo archive treated as untrusted.

Labels read with a whitelist unpickler (no code runs).

Arrays memory-mapped inside the zip.

Archive's TensorFlow model never loaded.

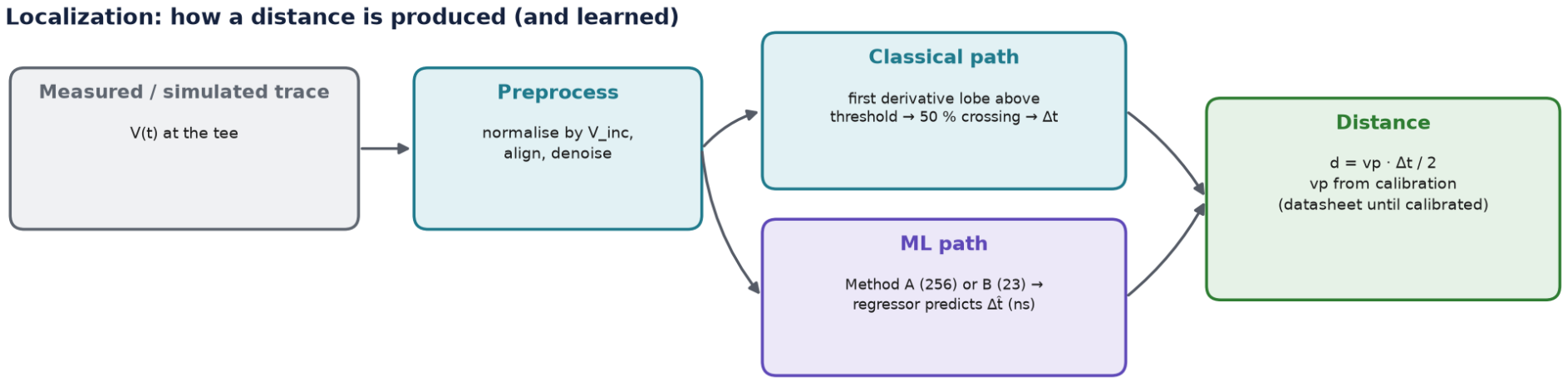
Python reads it in isolated mode (-I).

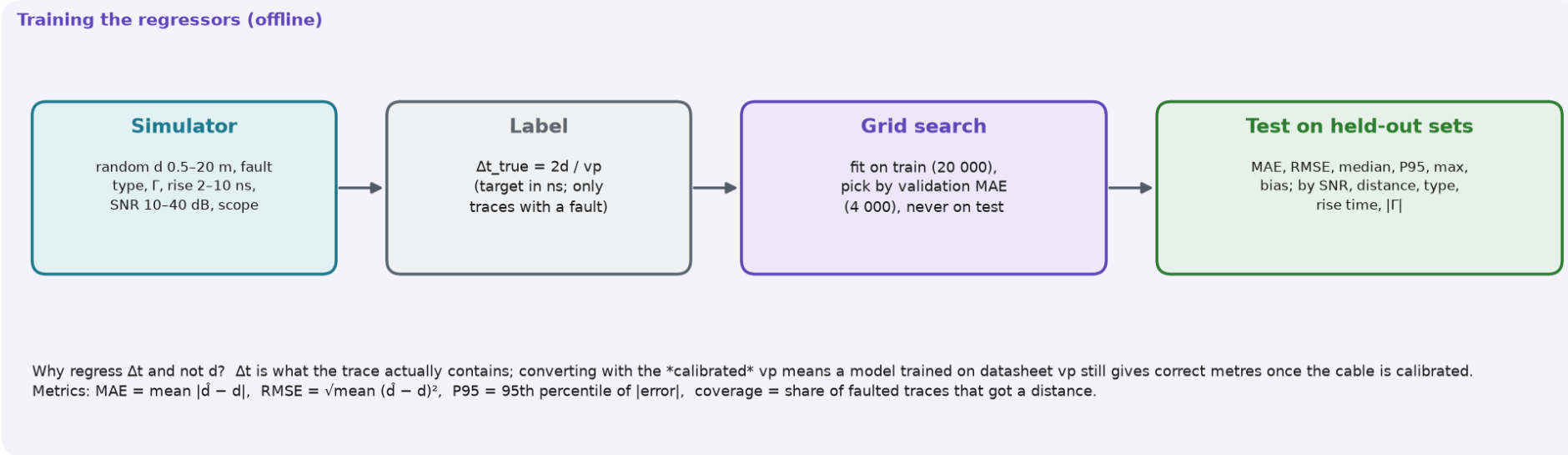
Part 6

Localization with ML / NN

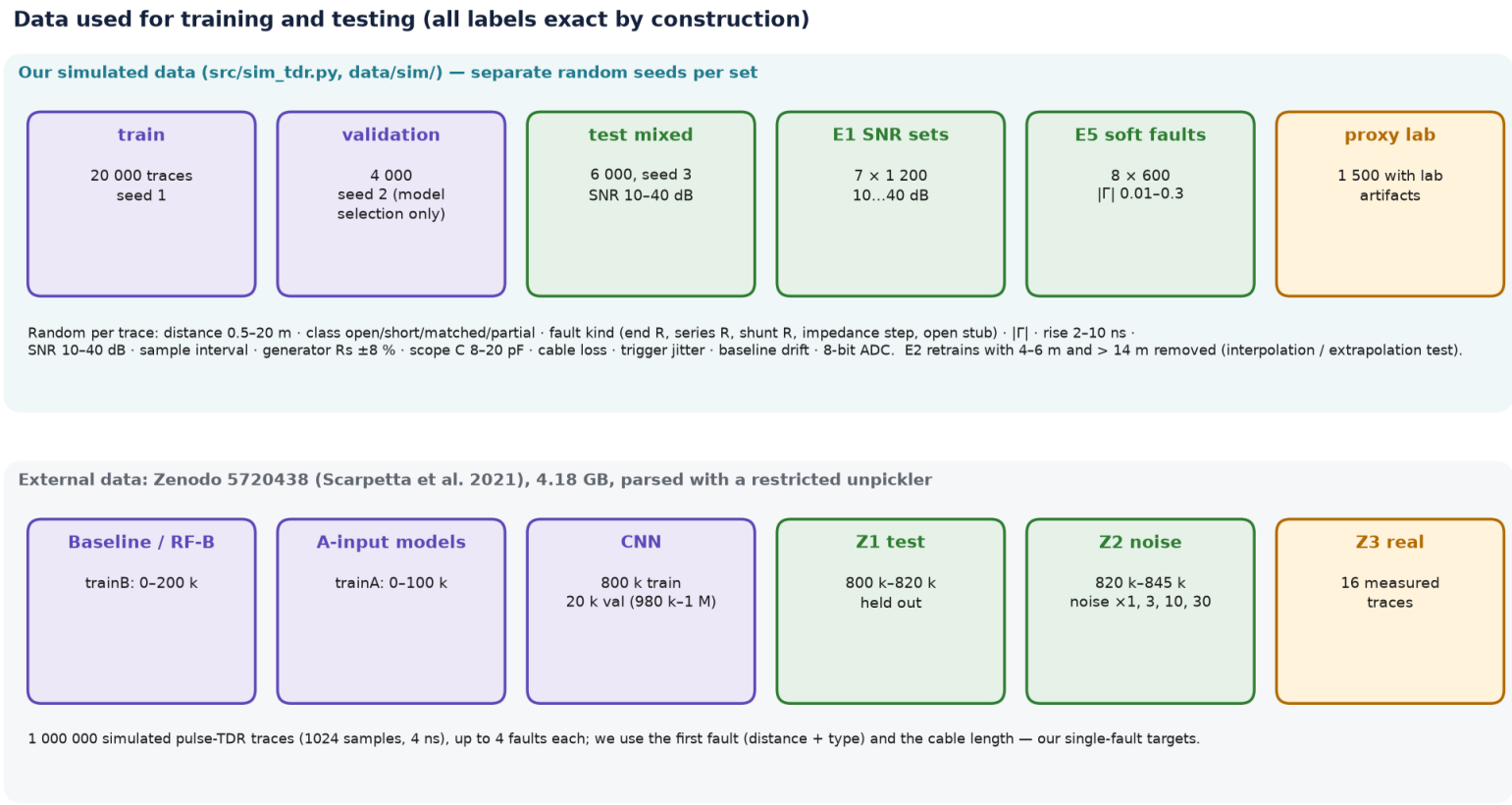
How far away is the fault? Classical timing vs five learned regressors

What localization is and how we do it





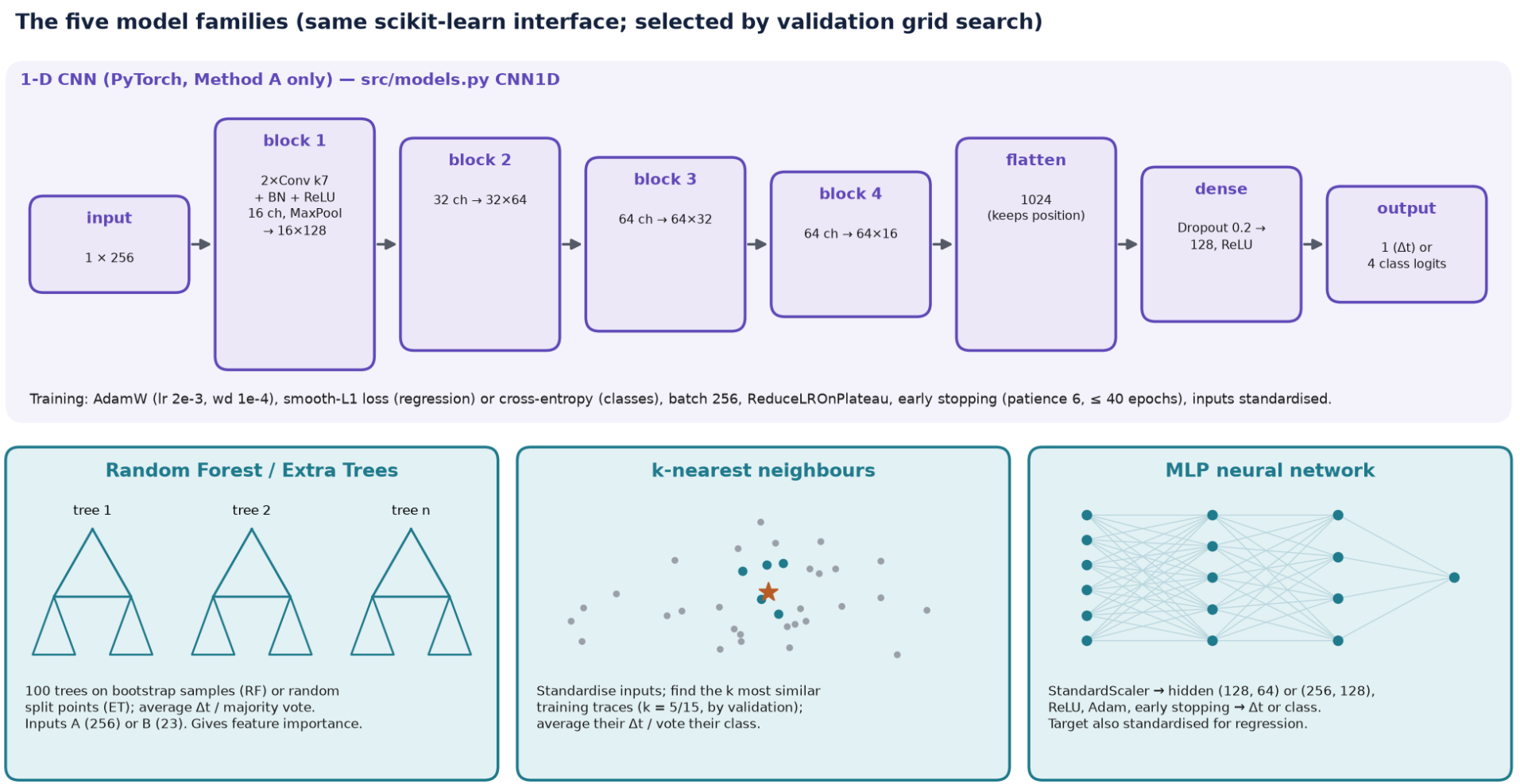
Training data and methodology



- **Train:** 20 000 simulated traces.
- **Validation:** 4 000, only for choosing hyper-parameters.
- **Test:** 6 000 mixed + E1 (7 × 1 200) + E5 (8 × 600).
- **Target:** Δt = 2d/vp (ns), traces with a fault only.
- **Excluded:** traces with no incident edge.

Real-data test: 16 measured traces from Zenodo 5720438 (Part 12).

The five model families and their architecture

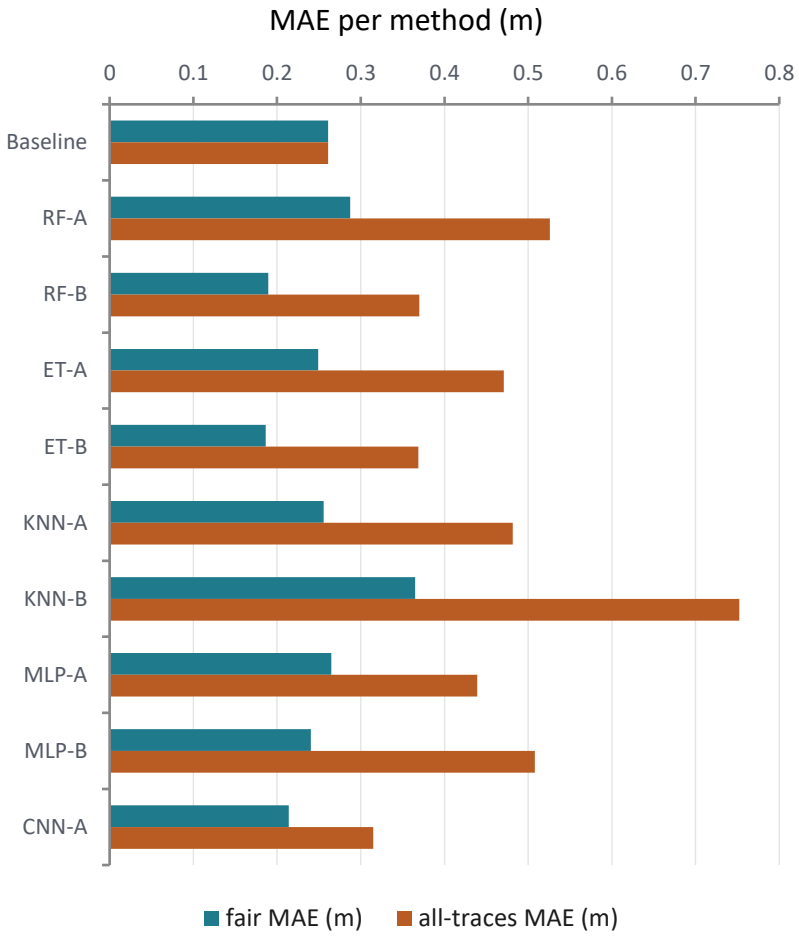


Selected model configuration (validation grid search)

Model	Task	Selected parameters	Val. score	Size (MB)	Detail
RF-A	distance	{max_features: sqrt, min_samples_leaf: 2}	4.79 ns	20.26	100 trees, 827,092 nodes
ET-A	distance	{max_features: 0.3, min_samples_leaf: 2}	4.02 ns	32.56	100 trees, 1,309,182 nodes
KNN-A	distance	{n_neighbors: 5, weights: distance}	4.13 ns	14.55	stores 15,035 training traces, k = 5
MLP-A	distance	{hidden_layer_sizes: (256, 128), alpha: 0.0001}	3.82 ns	1.90	98,817 weights, layers (256, 128)
CNN-A	distance	{epochs: 40, patience: 6}	2.64 ns	0.92	245,393 weights (1-D CNN)
RF-B	distance	{max_features: 0.5, min_samples_leaf: 2}	2.78 ns	19.64	100 trees, 828,246 nodes
ET-B	distance	{max_features: 1.0, min_samples_leaf: 2}	2.78 ns	31.55	100 trees, 1,303,340 nodes
KNN-B	distance	{n_neighbors: 15, weights: distance}	7.46 ns	1.08	stores 15,035 training traces, k = 15
MLP-B	distance	{hidden_layer_sizes: (128, 64), alpha: 0.0001}	4.75 ns	0.23	11,393 weights, layers (128, 64)
RF-A	class	{max_features: 0.3, min_samples_leaf: 1}	0.967	1.66	100 trees, 71,988 nodes
ET-A	class	{max_features: sqrt, min_samples_leaf: 1}	0.968	4.86	100 trees, 257,494 nodes
KNN-A	class	{n_neighbors: 5, weights: distance}	0.951	19.31	stores 20,000 training traces, k = 5
MLP-A	class	{hidden_layer_sizes: (256, 128), alpha: 0.0001}	0.974	1.12	99,204 weights, layers (256, 128)
CNN-A	class	{epochs: 40, patience: 6}	0.985	0.93	245,780 weights (1-D CNN)
RF-B	class	{max_features: 0.5, min_samples_leaf: 1}	0.979	1.44	100 trees, 67,676 nodes
ET-B	class	{max_features: 0.5, min_samples_leaf: 1}	0.979	3.69	100 trees, 205,776 nodes
KNN-B	class	{n_neighbors: 5, weights: distance}	0.953	1.19	stores 20,000 training traces, k = 5
MLP-B	class	{hidden_layer_sizes: (256, 128), alpha: 0.0001}	0.971	0.45	39,556 weights, layers (256, 128)

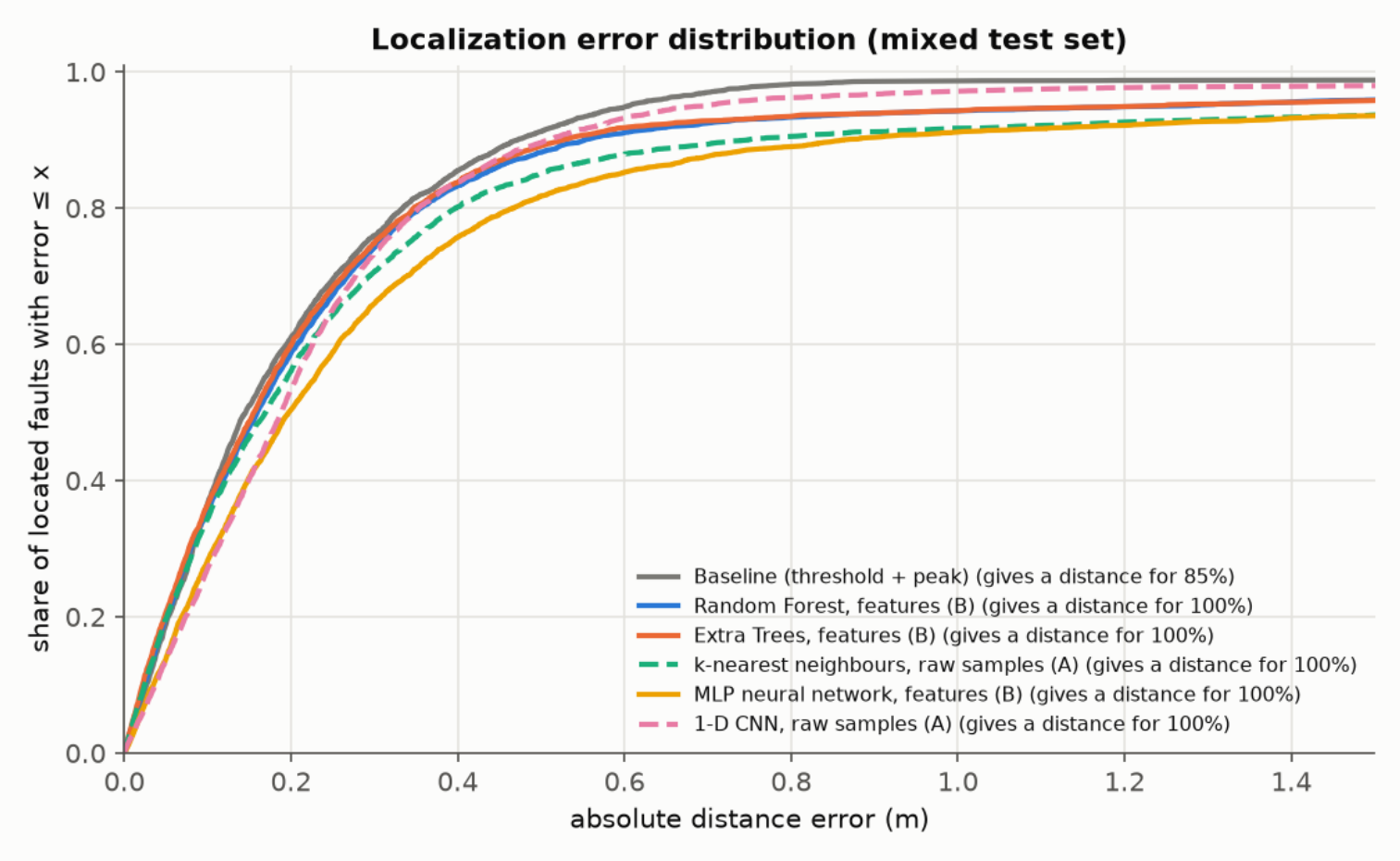
Localization results on the mixed test set

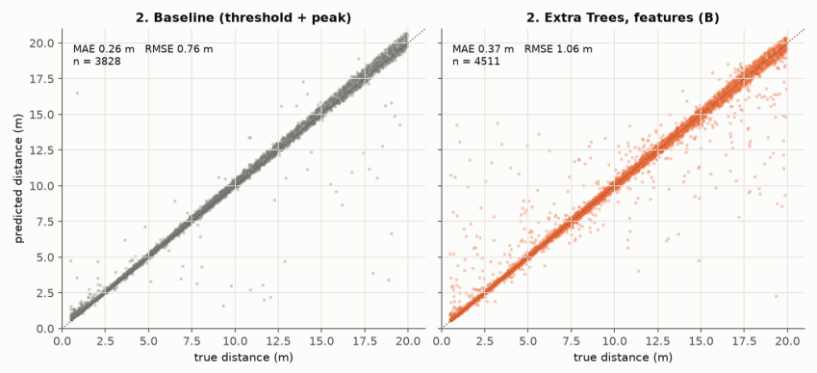
Method	Coverage	MAE fair (m)	MAE all (m)	Median (m)	P95 (m)	Max (m)	≤ 0.25 m
Baseline (classical)	85 %	0.26	0.26	0.15	0.61	15.59	69 %
Random Forest (raw)	100 %	0.29	0.53	0.17	2.68	11.87	63 %
Random Forest (features)	100 %	0.19	0.37	0.16	1.27	17.28	67 %
Extra Trees (raw)	100 %	0.25	0.47	0.16	2.13	12.12	66 %
Extra Trees (features)	100 %	0.19	0.37	0.16	1.23	17.11	68 %
k-NN (raw)	100 %	0.26	0.48	0.17	2.05	15.78	64 %
k-NN (features)	100 %	0.36	0.75	0.28	3.78	13.90	46 %
MLP (raw)	100 %	0.26	0.44	0.20	1.47	12.29	58 %
MLP (features)	100 %	0.24	0.51	0.20	2.07	16.29	59 %
1-D CNN (raw)	100 %	0.21	0.31	0.19	0.70	17.25	65 %



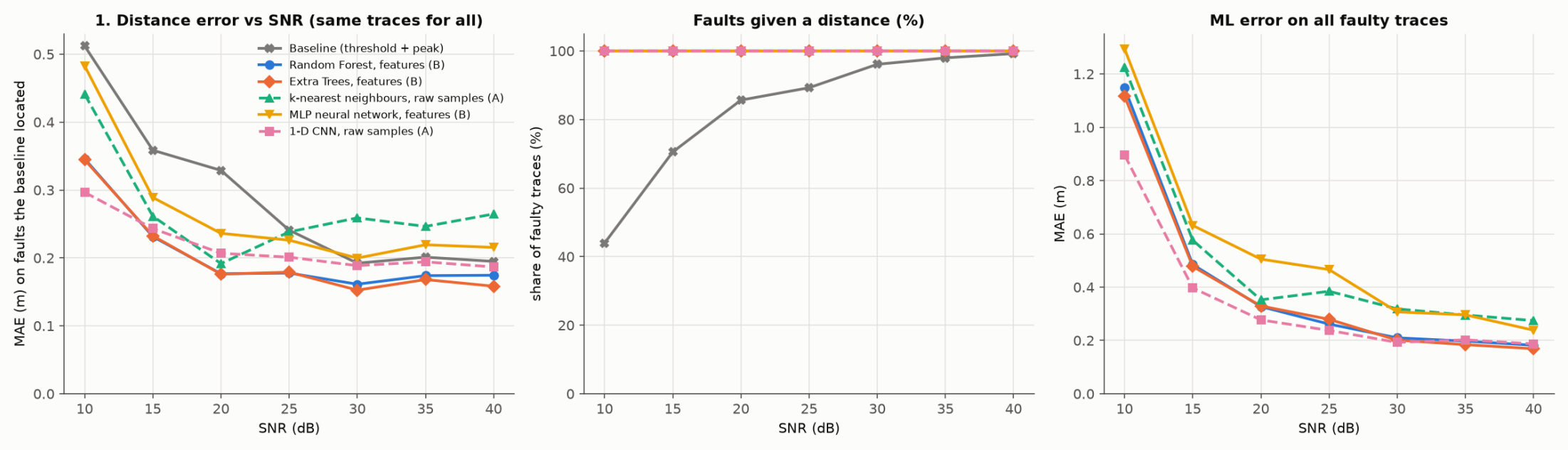
Best regressor (fair MAE): Extra Trees (features). The baseline is accurate when it answers but leaves 15 % of faults unlocated.

How the localization error is distributed





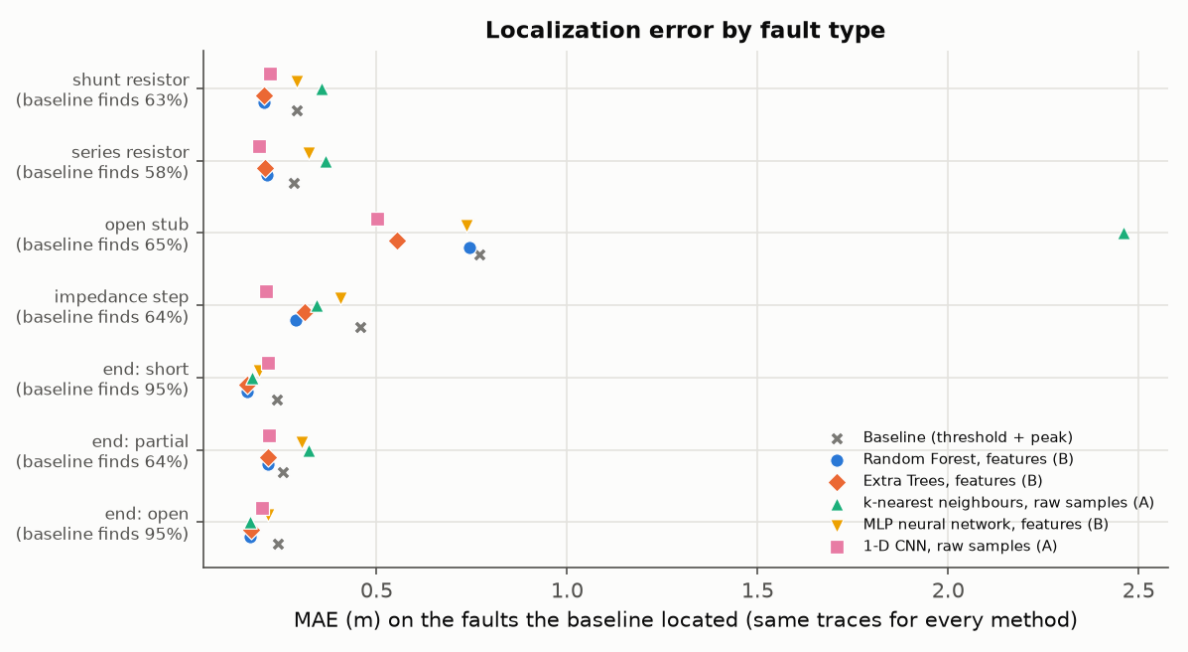
Accuracy vs noise (E1)

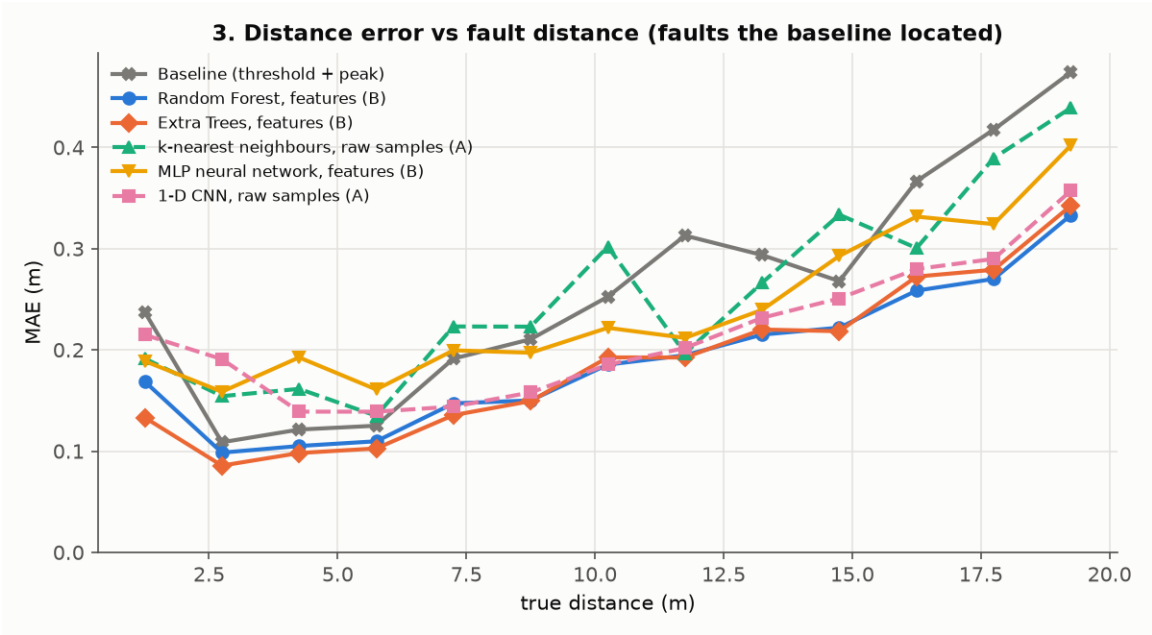


SNR (dB)	Baseline	Extra Trees B	CNN-A
10	0.51	0.34	0.30
15	0.36	0.23	0.24
20	0.33	0.18	0.21
25	0.24	0.18	0.20
30	0.19	0.15	0.19
35	0.20	0.17	0.19
40	0.19	0.16	0.19

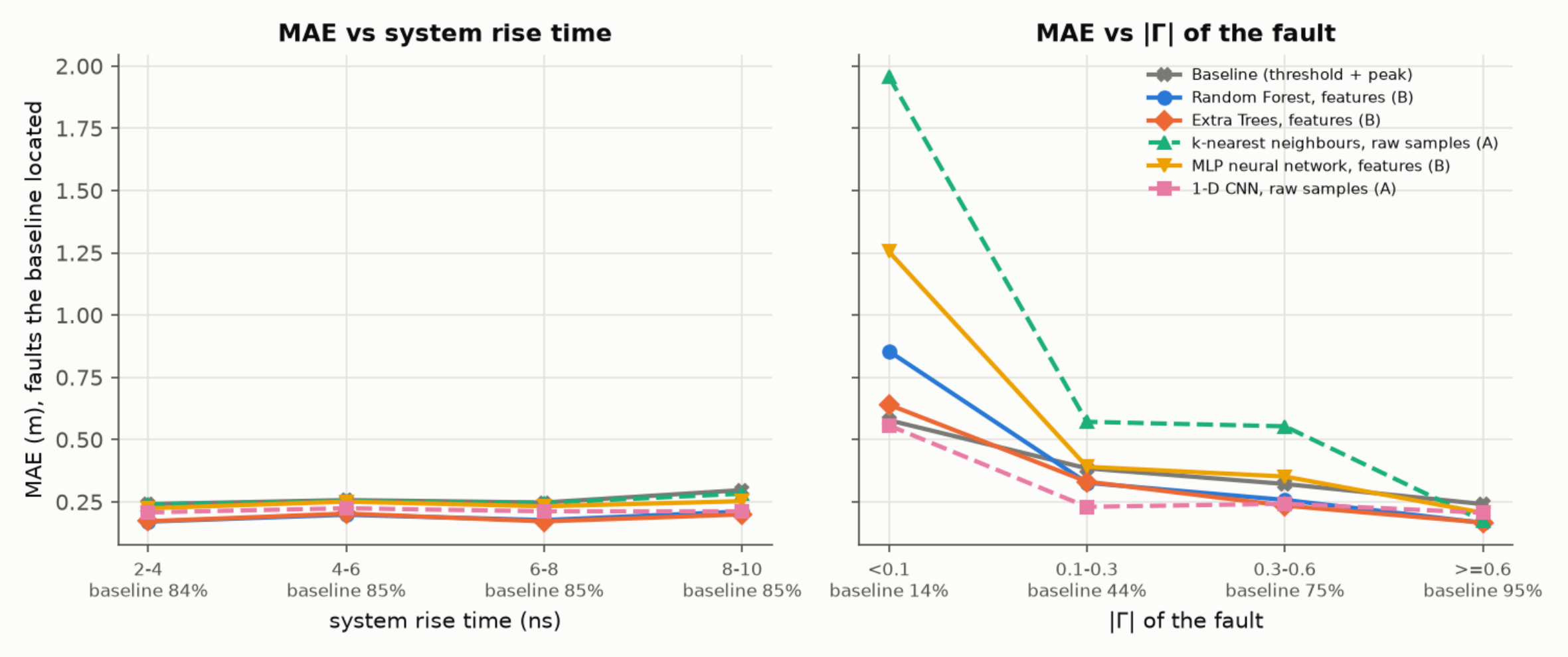
MAE (m) on faults the baseline located (E1, 1 200 traces per SNR). At 10 dB the baseline locates only 44 % of faults; the models answer all.

Which conditions make localization harder

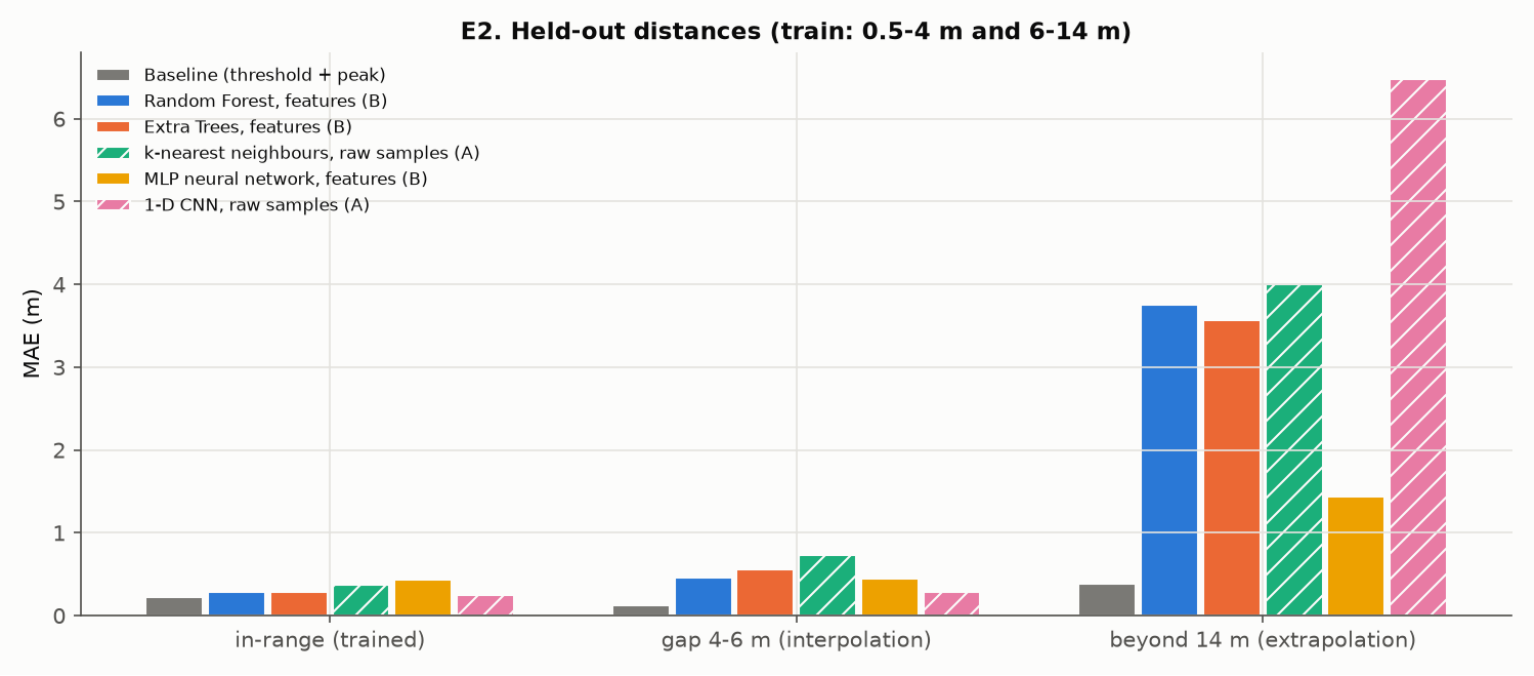




Rise time and fault strength



Generalisation to unseen distances (E2)



Band	Baseline	ET-B	CNN-A
in-range (trained)	0.22	0.29	0.25
gap 4-6 m (interpolation)	0.12	0.56	0.29
beyond 14 m (extrapolation)	0.38	3.57	6.48

Consequence: train on the full distance range of the cables you will test (limitation L3).

Worst cases of the best regressor (ET-B)

True d (m)	Predicted (m)	Error (m)	Fault	Γ	SNR (dB)	Rise (ns)	Baseline found?
19.35	2.25	-17.11	end: partial	0.38	11.58	5.48	no
0.89	14.24	13.36	impedance step	0.25	26.98	4.34	yes
1.05	13.06	12.00	open stub	-0.33	10.08	2.19	no
3.24	14.38	11.14	open stub	-0.33	16.93	8.67	no
19.69	8.55	-11.14	end: partial	-0.14	29.72	4.75	no
19.42	8.73	-10.68	end: partial	0.09	15.86	3.13	no
19.87	9.33	-10.54	open stub	-0.33	21.21	4.24	no
0.54	10.74	10.20	open stub	-0.33	23.58	8.76	no

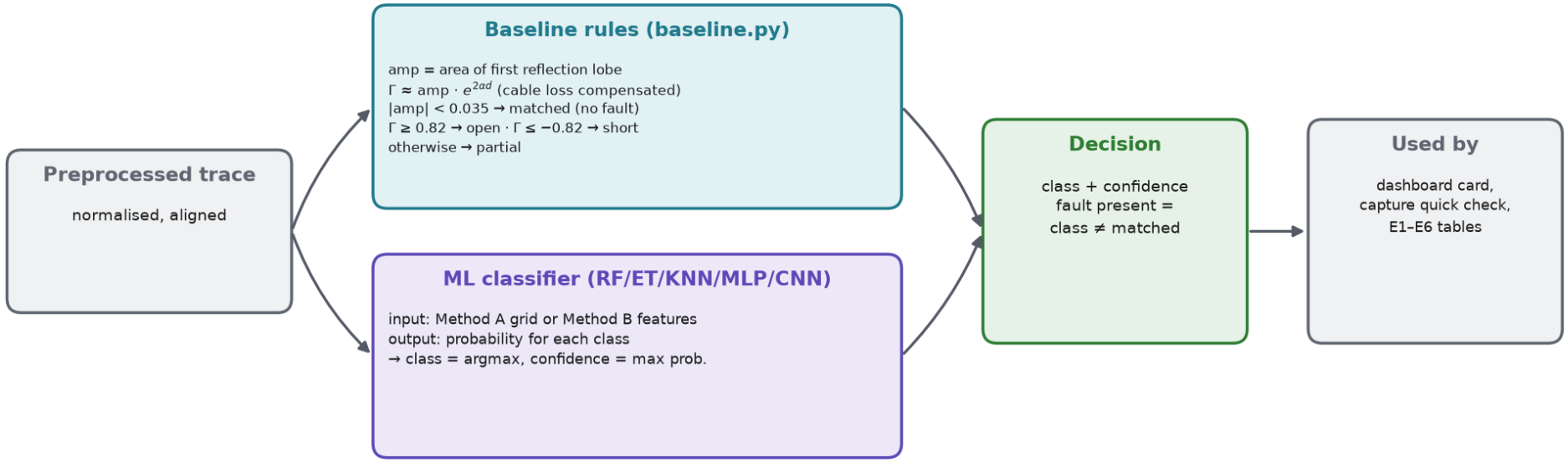
Part 7

Detection with ML / NN

Is there a fault, and what kind? Rules vs learned classifiers

What detection is and how we do it

Detection: is there a fault, and what kind? (4 classes: open · short · matched · partial)



How detection is scored (src/analysis.py)

Binary view (fault present?)

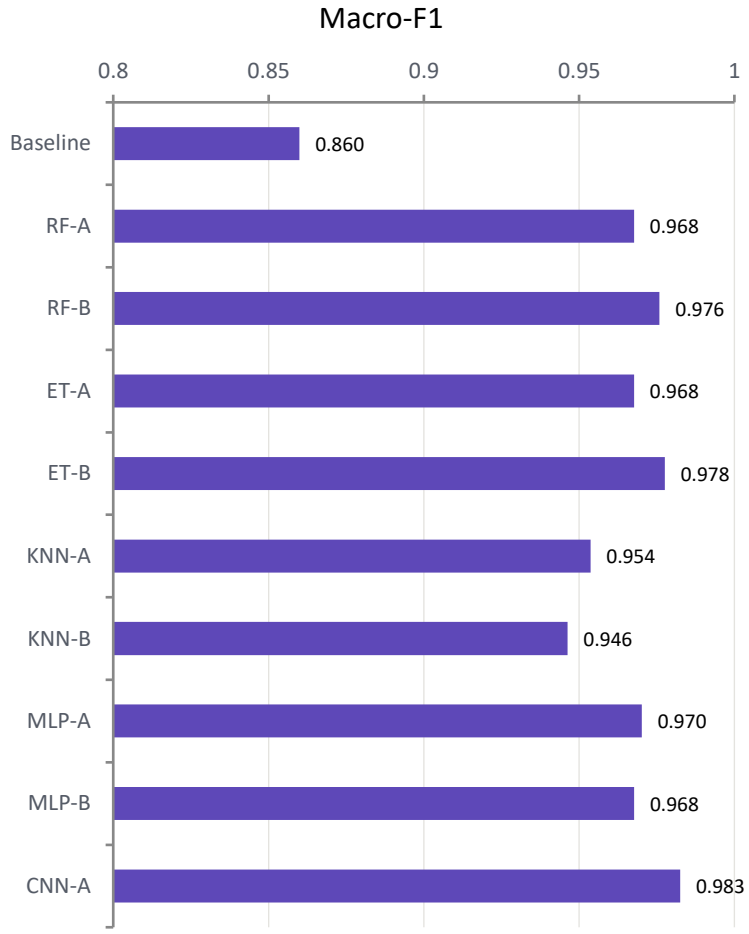
TP: fault and detected FP: matched line called a fault (false alarm)
FN: fault called matched (miss) TN: matched and called matched
precision = TP/(TP+FP) recall = TP/(TP+FN)
F1 = 2·P·R/(P+R) false-alarm rate = FP/(FP+TN)

Four-class view

accuracy = correct / all; per-class precision, recall, F1;
macro-F1 = mean of the four F1 (each class counts equally)
confusion matrix: rows = true class, columns = predicted
mAP / IoU do not apply (no bounding boxes: 1-D signal)

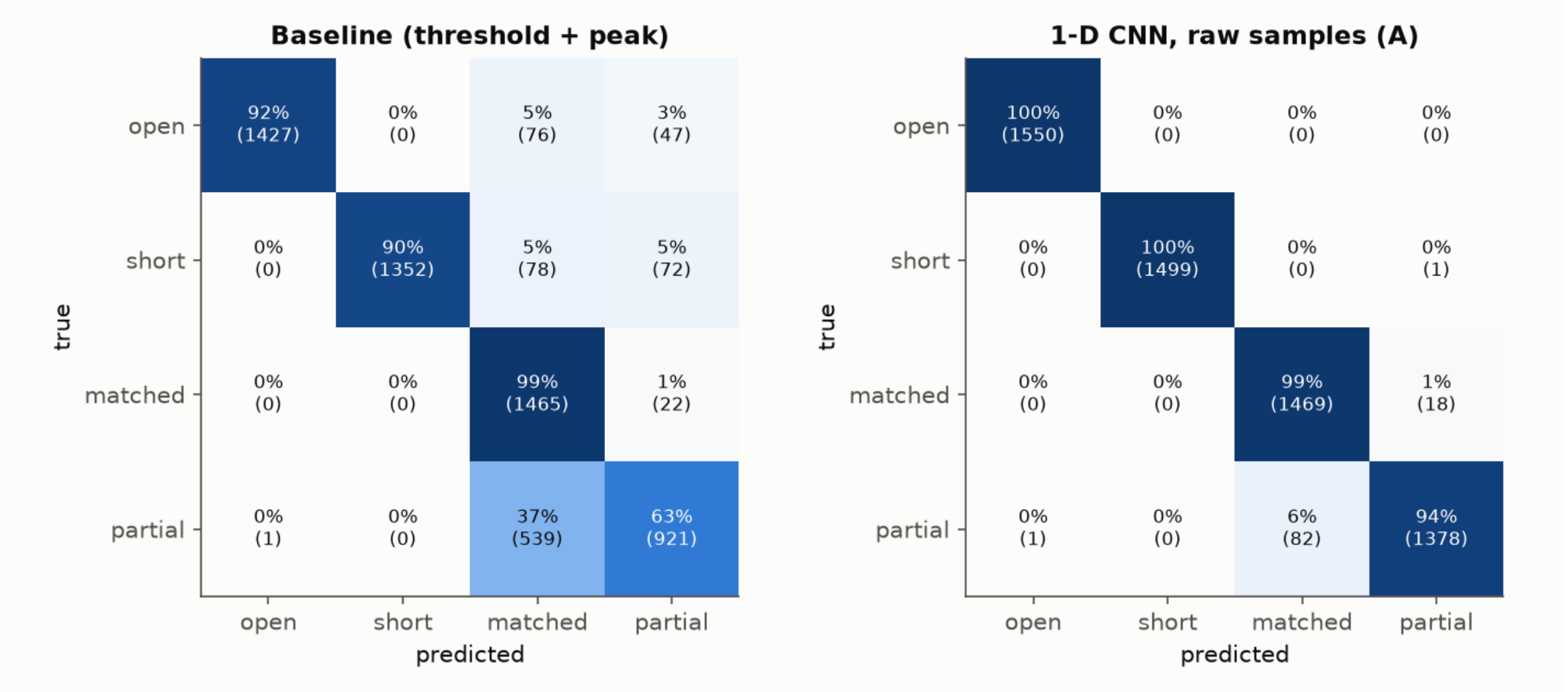
Detection results: binary and four-class

Method	Precision	Recall	F1 (binary)	False alarms	Misses	Accuracy (4-class)	Macro-F1
Baseline (classical)	0.994	0.846	0.914	1.5 %	15.4 %	0.861	0.860
Random Forest (raw)	0.992	0.968	0.980	2.4 %	3.2 %	0.968	0.968
Random Forest (features)	0.992	0.978	0.985	2.4 %	2.2 %	0.976	0.976
Extra Trees (raw)	0.995	0.964	0.979	1.5 %	3.6 %	0.968	0.968
Extra Trees (features)	0.994	0.977	0.985	1.7 %	2.3 %	0.978	0.978
k-NN (raw)	0.995	0.946	0.970	1.5 %	5.4 %	0.955	0.954
k-NN (features)	0.984	0.951	0.967	4.8 %	4.9 %	0.947	0.946
MLP (raw)	0.987	0.977	0.982	3.8 %	2.3 %	0.971	0.970
MLP (features)	0.987	0.972	0.980	3.9 %	2.8 %	0.968	0.968
1-D CNN (raw)	0.996	0.981	0.989	1.2 %	1.9 %	0.983	0.983



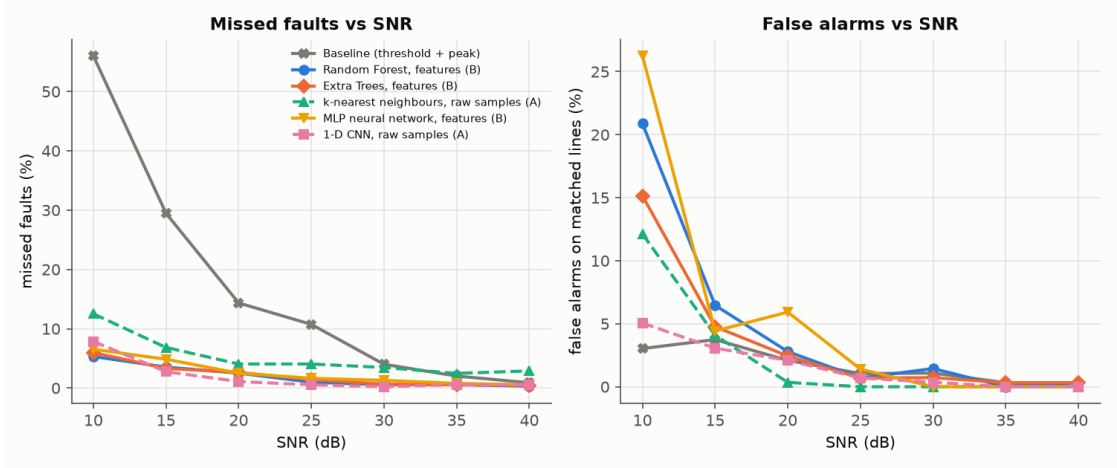
Best classifier: 1-D CNN (raw) (macro-F1 0.983) vs baseline 0.860.

Confusion matrices: baseline vs best classifier

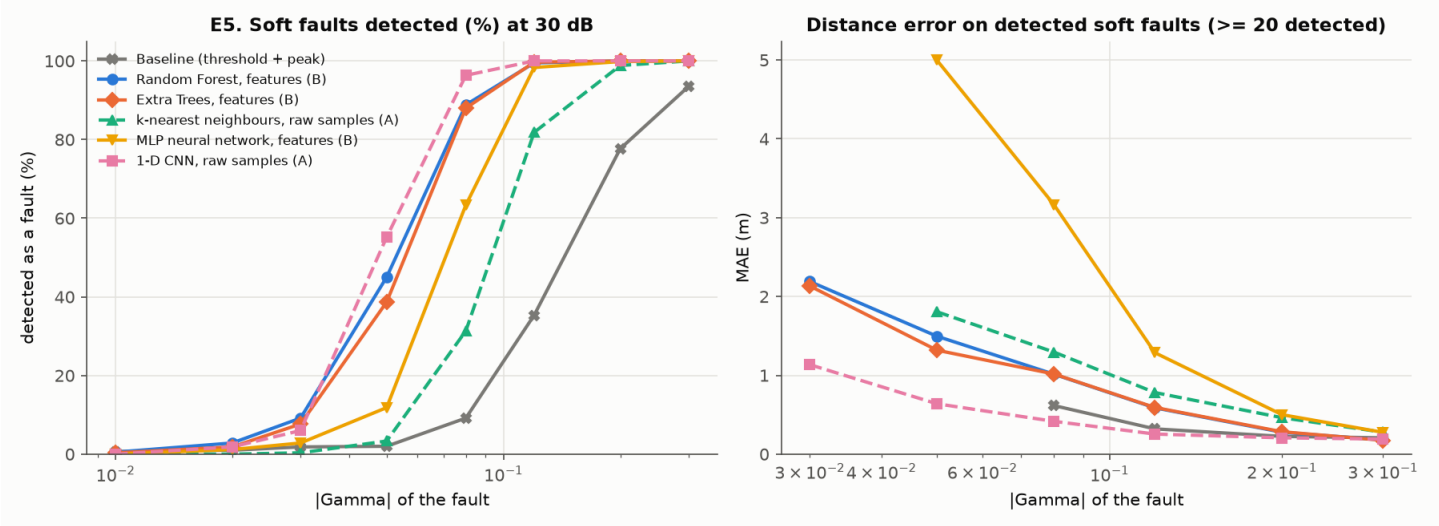


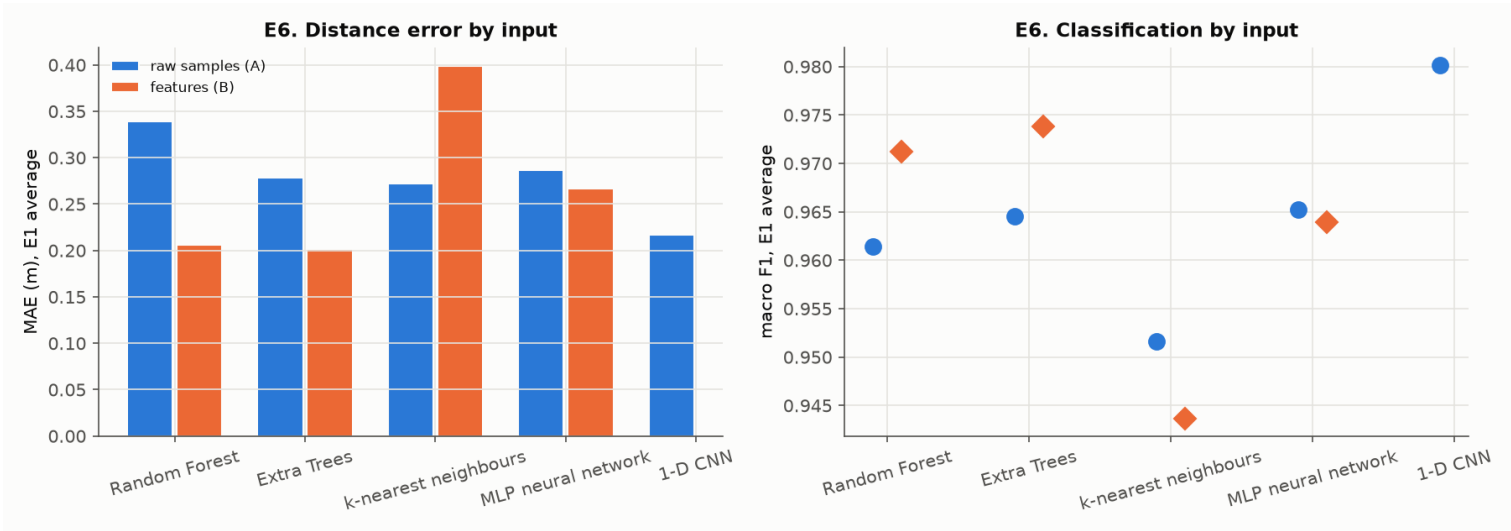
Misses and false alarms

Method	Kind	Condition	n	Rate
Baseline	miss rate	|Gamma| 0.1-0.3	426	56.1 %
Baseline	miss rate	|Gamma| 0.3-0.6	755	24.9 %
Baseline	miss rate	|Gamma| <0.1	95	87.4 %
Baseline	miss rate	|Gamma| >=0.6	3237	5.7 %
Baseline	miss rate	SNR (dB) 10-15	802	40.4 %
Baseline	miss rate	SNR (dB) 15-20	741	22.7 %
Baseline	miss rate	SNR (dB) 20-30	1484	10.0 %
Baseline	miss rate	SNR (dB) 30-40	1486	3.5 %
Baseline	false-alarm rate	SNR (dB) 10-15	248	1.6 %
Baseline	false-alarm rate	SNR (dB) 15-20	255	0.8 %
Baseline	false-alarm rate	SNR (dB) 20-30	507	2.6 %
Baseline	false-alarm rate	SNR (dB) 30-40	477	0.6 %
CNN-A	miss rate	|Gamma| 0.1-0.3	426	4.2 %
CNN-A	miss rate	|Gamma| 0.3-0.6	755	4.2 %



Weak (soft) faults and the input representation



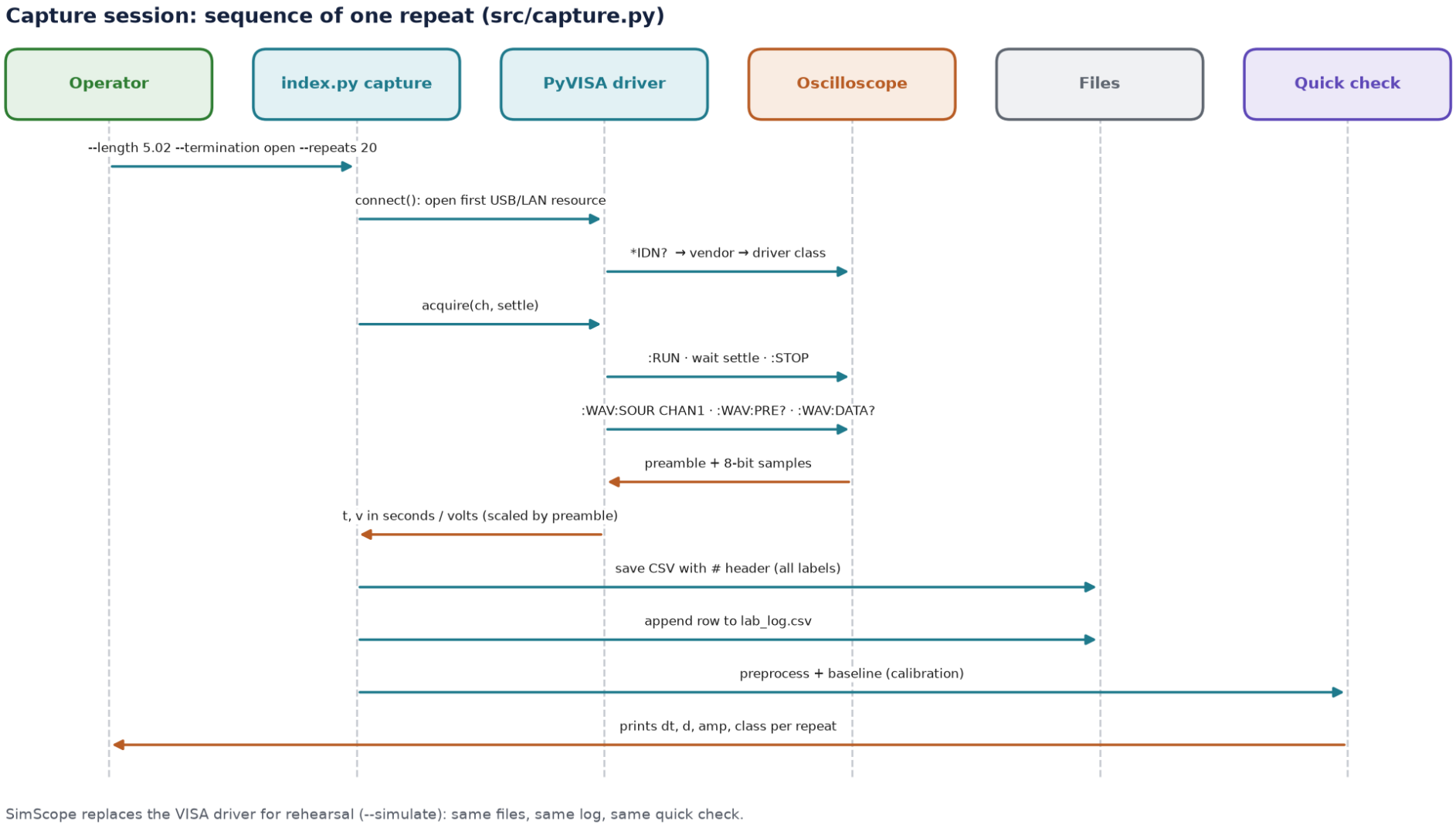


Part 8–9

End-to-end working and hardware–software integration

What happens, step by step, from power-on to a located fault

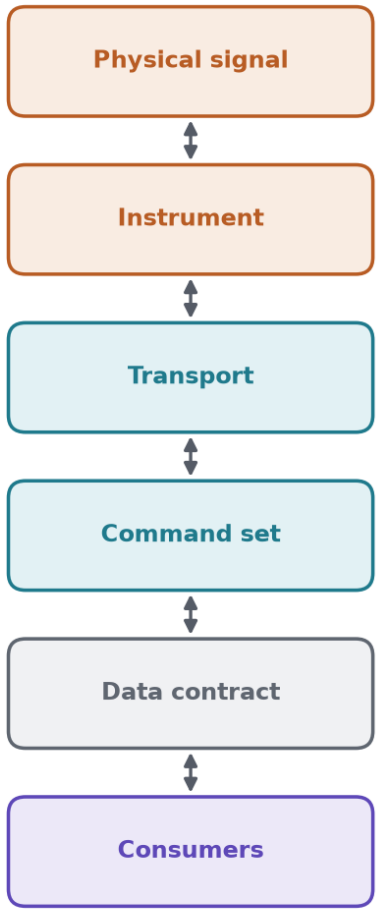
Capture session: what happens in one repeat



SimScope replaces the VISA driver for rehearsal (--simulate): same files, same log, same quick check.

Integration layer by layer

Hardware ↔ software integration, layer by layer



step + echo on 50 Ω coax; BNC tee at CH1 defines d = 0

check: tee directly on CH1, short lead, 1 MΩ input

scope samples CH1 (8-bit, ≥ 500 MSa/s), holds the record

check: trigger Normal at 50 %, BW limit off, other channels off

USB-TMC (or LAN/VXI-11) through VISA: NI-VISA or pyvisa-py

*check: capture --list shows the *IDN? string*

SCPI per vendor: Rigol :WAV, Keysight :WAVeform, Tek CURVE?, Siglent WF?

check: preamble → volts and seconds; unknown vendor → Keysight style

CSV with # header (labels) + lab_log.csv (28 columns) + calibration.json

check: universal reader also parses scope-exported CSVs

calibrate · lab-dataset · fit · train · evaluate · analyze · dashboard

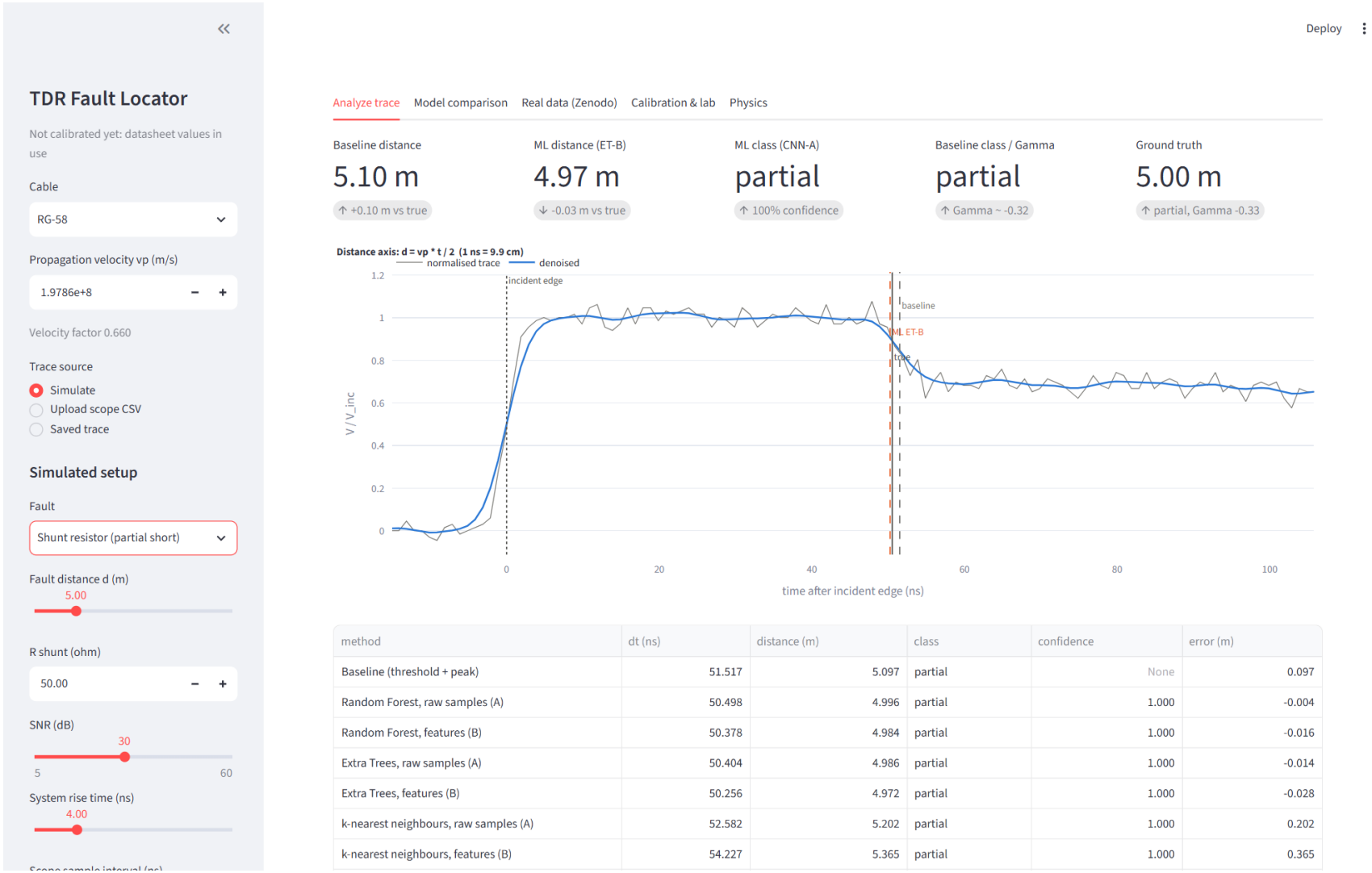
check: selftest runs this whole chain with SimScope

Synchronisation: the scope's trigger defines t = 0; software re-aligns every trace on the incident 50 % crossing, so trigger jitter and delay do not bias Δt.

Real execution example (self-test, simulated scope)

Feature	Actual output
Capture session with logging (capture --simulate)	28 traces + log rows written; quick-check distances: 2 m open -> 2.04 m; 5 m open -> 5.07 m; 10 m open -> 10.11 m; 5 m short -> 5.06 m; 5 m partial 100 ohm -> 5.07 m; 5 m partial 22 ohm -> 5.05 m
Read any scope CSV (inspect)	exit 0; vendor/format detected; baseline d = 5.065 m (true 5.00); plot saved = True
Velocity calibration (calibrate)	vp = 1.9616e+08 m/s vs simulated truth 1.9786e+08 (-0.86 %); t_offset 0.23 ns; rise 4.15 ns; resolution 0.41 m; SNR 30.5 dB
Simulator-to-measurement fit (fit)	exit 0; RMSE 0.0238 (normalised); fitted vp 1.972e+08 m/s; overlay figure written
Lab dataset builder (lab-dataset)	exit 0; 28 traces processed; Method A matrix (28, 256)

Example: one trace through the dashboard



method	dt (ns)	distance (m)	class	confidence	error (m)
Baseline (threshold + peak)	51.517	5.097	partial	None	0.097
Random Forest, raw samples (A)	50.498	4.996	partial	1.000	-0.004
Random Forest, features (B)	50.378	4.984	partial	1.000	-0.016
Extra Trees, raw samples (A)	50.404	4.986	partial	1.000	-0.014
Extra Trees, features (B)	50.256	4.972	partial	1.000	-0.028
k-nearest neighbours, raw samples (A)	52.582	5.202	partial	1.000	0.202
k-nearest neighbours, features (B)	54.227	5.365	partial	1.000	0.365

Part 10–11

Features, testing and validation

Every feature executed for real, with expected vs actual output

Testing strategy

Unit

62 pytest tests:

physics levels and delays

baseline on clean traces

every scope CSV dialect

every model family

safe unpickler, zip memmap

16 regression tests for review findings

Integration

capture → calibrate → fit → lab-dataset

evaluate with a lab log (E3/E4 = LAB)

dashboard driven through all inputs

System / performance

full self-test (19 features)

latency per model

Zenodo pipeline on 1 M traces

real-trace test (16)

Robustness / edge cases

7 wrong CLI calls

low amplitude traces

2 traces only, zero noise

blank channel cells

relative paths

Feature test matrix (1/3)

ID	Feature	Expected	Actual	Status
F01	Instrument discovery (capture --list)	Exit 0 and a clear 'no instrument' message (no hardware attached here)	exit 0; No instruments found (Visa Library at py). Check the USB/LAN cable, that the scope is on, its USB mode (USBTMC) and the VISA driver; see hardware/HARDWARE_GUIDE.md, H2.	Pass
F02	Capture session with logging (capture --simulate)	28 CSV files + 28 log rows; live distance within 0.3 m of the cable length	28 traces + log rows written; quick-check distances: 2 m open -> 2.04 m; 5 m open -> 5.07 m; 10 m open -> 10.11 m; 5 m short -> 5.06 m; 5 m partial 100 ohm -> 5.07 m; 5 m partial 22 ohm -> 5.05 m	Pass
F03	Read any scope CSV (inspect)	Distance within 0.3 m of 5 m, class open, PNG written	exit 0; vendor/format detected; baseline d = 5.065 m (true 5.00); plot saved = True	Pass
F04	Velocity calibration (calibrate)	vp within 1.5 % of the simulator's true vp; calibration.json + fig0 written	vp = 1.9616e+08 m/s vs simulated truth 1.9786e+08 (-0.86 %); t_offset 0.23 ns; rise 4.15 ns; resolution 0.41 m; SNR 30.5 dB	Pass
F05	Simulator-to-measurement fit (fit)	Converges, normalised RMSE < 0.05	exit 0; RMSE 0.0238 (normalised); fitted vp 1.972e+08 m/s; overlay figure written	Pass
F06	Lab dataset builder (lab-dataset)	data/processed/lab.csv with 28 rows and a (28, 256) Method A matrix	exit 0; 28 traces processed; Method A matrix (28, 256)	Pass
F07	Synthetic dataset, transmission-line model (simulate)	400 rows, X (400, 256), all 4 classes, finite values	exit 0; 400 labelled traces, X (400, 256), classes {'matched': 108, 'open': 100, 'short': 99, 'partial': 93}, raw traces saved	Pass

Feature test matrix (2/3)

ID	Feature	Expected	Actual	Status
F08	Synthetic dataset, brief's simple model + lab artifacts	200 rows written, source tagged as artifacts	exit 0; 200 rows, source column = sim_artifacts	Pass
F09	Physics check of the simulator	Every level within 0.03 of theory (small cable loss allowed)	final level / theory 1+Gamma: open: 1.985/2.000; short: 0.015/0.000; matched gamma=0: 1.000/1.000; partial zl=100: 1.328/1.333; partial zl=22: 0.617/0.611; partial kind=1,r_fault=50: 1.328/1.333; partial kind=2,r_fault=50: 0.672/0	Pass
F10	Classical baseline detector	At 40 dB: median distance error < 0.25 m (half the 0.4-0.5 m resolution of a 4-5 ns edge) and class accuracy > 0.9	SNR 40 dB: median |err| 0.17 m, mean 0.20 m, class acc 0.99, missed 1%; SNR 20 dB: median |err| 0.16 m, mean 0.30 m, class acc 0.81, missed 21%	Pass
F11	Model training with validation grid search (train)	18 models in models/tdr_models.joblib and a validation_grid.csv	exit 0; 18 models stored (RF/ET/KNN/MLP x A/B + CNN-A, regression + classification); best val MAE ET-B 3.36 ns; best val macro-F1 ET-B 0.977	Pass
F12	Inference latency (all models)	Well under 1 s per trace end to end (interactive use)	front end (preprocess + baseline + 23 features) 0.7 ms/trace; single-trace model call 0.62-22.8 ms (slowest KNN-A reg); batched 0.008-0.24 ms/trace	Pass
F13	Experiments E1-E6 + figures (evaluate --quick)	Exit 0, fig1..fig10 and summary.md written; E3/E4 run on the logged traces	exit 0; 11 figures, summary.md written; E3/E4 data source captured (lab-format) traces; best regressor ET-B, best classifier ET-B	Pass
F14	Dashboard: every input path (Streamlit AppTest)	No exception on any run, 5 tabs rendered	exit 0; 9 script runs (8 fault types + saved lab trace), no exceptions; tabs 5	Pass

Feature test matrix (3/3)

ID	Feature	Expected	Actual	Status
F15	Dashboard server start-up (index.py dashboard)	Health endpoint answers 'ok' and the page is served	health endpoint 'ok', index page 6467 bytes	Pass
F16	Error handling of the CLI	Non-zero exit (or a clear 'nothing to do' message) and no traceback	inspect missing file: exit 1, clean message; inspect garbage CSV: exit 1, clean message; capture without length: exit 2, clean message; calibrate without log: exit 2, clean message; fit missing file: exit 1, clean message; lab-dat	Pass
F17	Hardware circuit + expected-waveform generator	13 schematics (SVG + PNG) and 11 expected-waveform PNGs	exit 0; 13 circuit SVGs, 26 PNGs (circuits + expected waveforms)	Pass
F18	Unit test suite (pytest)	All tests pass	62 passed in 14.03s	Pass
F19	External real-data pipeline (Zenodo 5720438)	models/zenodo_models.joblib and the Z1/Z2/Z3 tables exist	all result files present; Z1 table has 8 rows (see ML_ANALYSIS.md)	Pass

Independent code review: findings and fixes

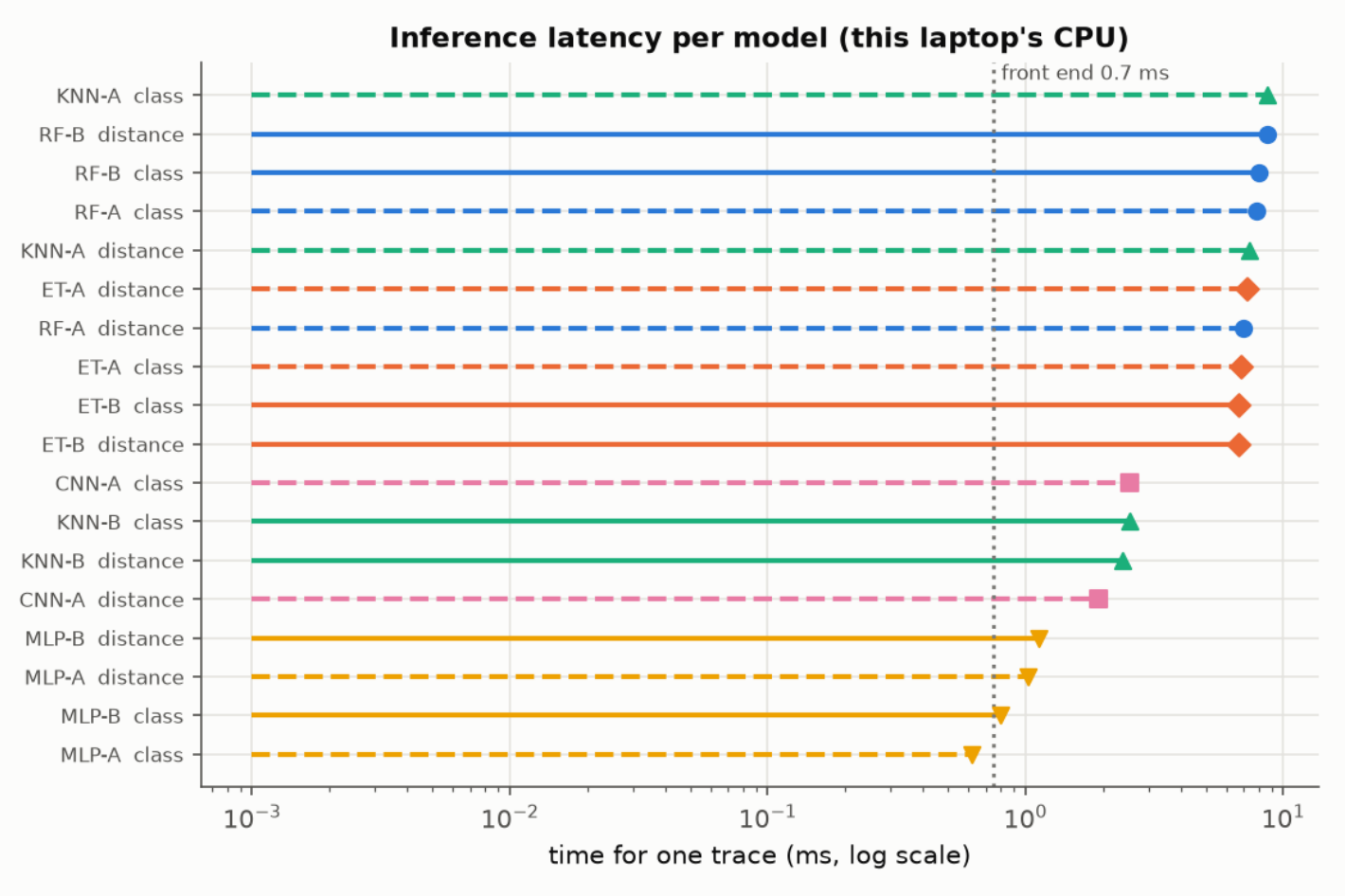
#	Finding	Effect before the fix	Fix
1	one calibration V_inc normalised every lab trace	low-amplitude traces misclassified	V_inc per trace, calibration as reference
2	relative capture paths unresolvable	lab log silently unusable	paths relative to the log, POSIX
3	fixture offset t0 removed only from Δt	~10 cm bias in model inputs	t0 removed from both model inputs
4	blank channel cell → float channel	lab files skipped	integral floats accepted
5	test overwrote a real report figure	fake calibration in dashboard	tests run in a temp TDR_HOME
6	calibrate crashed on 2 traces / zero noise	traceback	guarded fit and SNR
7	failed trace encoded like a short	confident wrong answers	no prediction, counted as not covered
8	Zenodo test noise = training noise	biased k-NN result	noise seed per split
9–12	options ignored, stale caches, Siglent delay, crash paths	inconsistent runs	bundle settings, cache fingerprint, TRDL, handlers

Part 12

Performance analysis

Speed, resources, real data and the objectives check

Speed: inference latency per model



0.75 ms

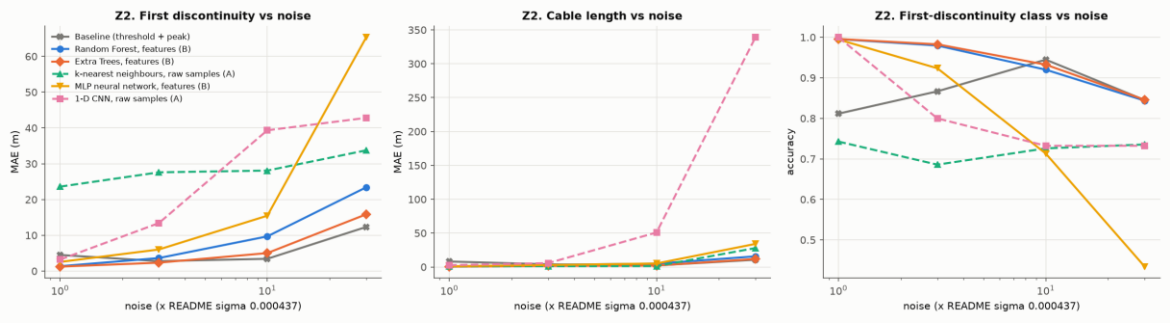
front end per trace (preprocess + baseline + features)

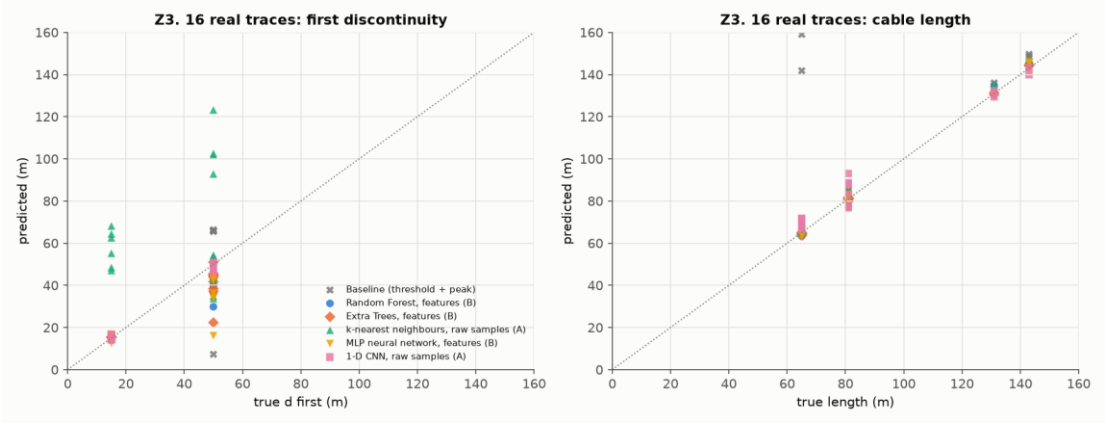
8.67 ms

slowest single-trace model call (KNN-A class)

- Training all 18 models: 44 min wall time on 16 cores while sharing the CPU (≈ 20 min of model fitting alone; the CNN is the largest share).
- E1–E6 evaluation: 31 min.

Real measured data: Zenodo 5720438

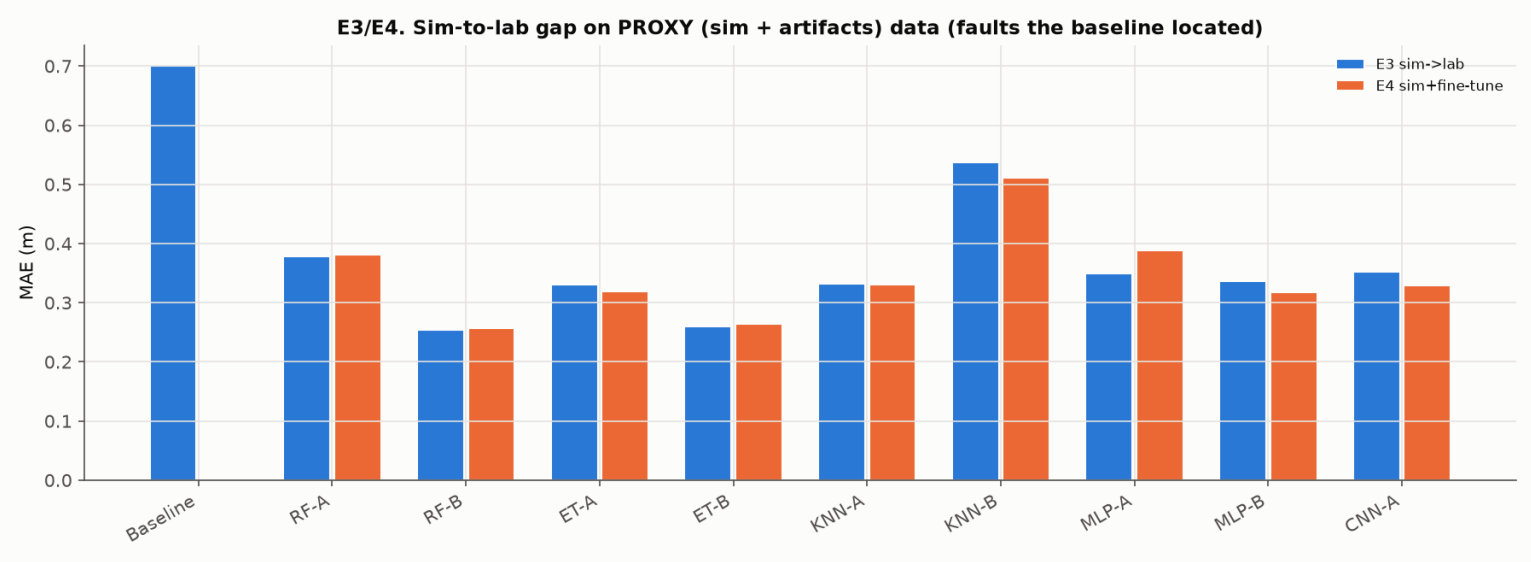




Model	Z1 sim: first fault MAE (m)	Z1 sim: length MAE (m)	Z3 REAL: first fault MAE (m)	Z3 REAL: length MAE (m)	Z3 REAL: type correct
Matched filter	4.40	8.35	8.64	13.16	9/16
ET-B	1.23	0.50	5.62	0.61	16/16
RF-B	1.35	0.63	5.60	1.09	16/16
CNN-A	3.22	2.63	1.23	4.22	16/16
ET-A *	2.82	0.47	1.98	0.90	16/16
RF-A *	3.12	0.80	3.53	1.31	16/16
MLP-A	5.32	0.55	4.53	2.34	16/16
MLP-B	2.49	1.00	7.17	1.17	16/16
KNN-B	3.18	2.07	8.10	3.11	16/16
KNN-A	23.84	0.92	32.73	1.58	8/16

* raw-input forests evaluated in the first run (Z1 noise shared with training; the seed check shows < 2 % effect).

Sim-to-lab gap (E3/E4, proxy until the lab session)



Reading the proxy result

Data: PROXY (sim + artifacts) (1 351 test traces).

E3: trained on clean simulation, tested with lab-like artifacts: every method gets worse (baseline 0.70 m).

On the faults the baseline locates, ML stays at about 0.25-0.35 m.

E4: +149 labelled proxy traces (×5 weight) changes MAE by ≤ 0.05 m: not enough.

Switches to LAB automatically once ≥ 20 logged traces exist.

Objectives vs measured results

Objective	Measured evidence	Verdict
Realistic labelled simulation	levels within 0.015 of 1 + Γ; delay exact (F09)	**met**
Baseline to beat	MAE 0.26 m on 85 % coverage	**met**
ML beats baseline	ET-B: MAE 0.37 m on all faults; fair 0.19 vs 0.26 m	**met (simulation)**
Fault type	CNN-A macro-F1 0.983 vs baseline 0.860	**met (simulation)**
Real data	16 measured Zenodo traces (Z3)	**measured**
Own lab data + calibration	procedure and software ready; rehearsed	**pending lab**
Interactive demo	dashboard, 5 tabs, tested (F14, F15)	**met**

Part 13

Problems, debugging and solutions

What went wrong, why, and how we know it is fixed

Problems found during development

Problem	Root cause	Solution	Verified by
Incident tail detected as an echo	lobe boundary inside the incident edge	echo lobe may not start before the incident lobe ends	baseline unit tests
False alarms at low SNR	noise from the pre-trigger only; fixed smoothing	MAD noise, adaptive Gaussian, longer pre-trigger	E1 false-alarm rates
Γ over-compensated	loss evaluated at the edge knee	loss at 0.1/rise	physics tests
Zenodo download broke (curl 56)	unstable connection	resumable retries, robust range reads	4.18 GB verified
Pulse tail triggered events	slow tail of the transmitted pulse	high-pass before correlating, blanking	Z1 baseline
Feature windows moved with t0	windows anchored on shifted axis	windows anchored on the incident edge	test_t_offset_shifts_every_time_feature
E2 lost its training set after cache rename	name changed, loader not	training-set name stored in bundle	selftest F13

Before and after: amplitude-independent normalisation

Before (one stored V_inc)

0.5 V generator: open → 'no reflection' / matched.

1.2 V: open and short → 'partial'.

3.0 V: merged-edge path, d = 0.29 m.

Calibration V_inc averaged over all open rows (low-SNR rows pulled it down).

After (V_inc per trace)

Estimated from each trace; calibration used when they agree.

All 8 amplitude/class cases: correct class,
$|d - 5| < 0.3$ m.

Calibration uses only normal-amplitude rows for V_inc.

Pinned by test_vinc_per_trace_any_amplitude
(8 cases).

Part 14–16

Achievements, limitations, future and conclusion

Separating what is done from what is pending

Status: completed, partial, pending, unverified

Completed

Simulator, baseline, features

18 models, E1–E6, analysis

Dashboard (5 tabs)

Capture, CSV reader, calibration, fit

Zenodo pipeline + 16 real traces

62 tests + 19-feature self-test

Hardware manual + 13 circuits

Partial

E3/E4 on proxy data (switches to LAB automatically)

Report draft: lab sections empty

Pending (needs lab)

Instructor approvals (S1)

Real calibration (S8)

≈ 180 + fault traces (S9–S11)

Retrain on calibration, E3/E4 LAB

Setup photo, demo rehearsal

Unverified

Scope drivers on a real instrument (written per vendor docs)

Fast-edge pulser (optional, not built)

Key results in numbers

0.37 m

Extra Trees (features): mean distance error on every faulted test trace (simulation)

85 %

share of faults the classical baseline located at all

0.983

1-D CNN (raw): macro-F1 over 4 classes

19/19

features passing the end-to-end self-test

62

unit and regression tests passing

16

real measured traces tested (Zenodo Z3)

Limitations

#	Limitation	Consequence / mitigation
L1	no own lab data yet	E3/E4 are proxy; switch to LAB after the session
L2	resolution ≈ vp·t_r/2	faults < 0.4–1.5 m from the tee merge; pulser helps
L3	learned models do not extrapolate (E2)	train on the full distance range used
L4	very weak faults (|Γ| < 0.05)	inconsistent detection by every method (E5)
L5	simulation fidelity (connectors, scope front end)	fit_sim and calibration narrow the gap; Zenodo shows a real gap
L6	single fault per trace in our own data	multi-fault only in Zenodo labels
L7	Zenodo k-NN stores 100 k raw traces per target	fine for evaluation, too large to deploy

Future scope

Better models

Domain randomisation: train with lab artifacts on.

Hybrid: ML inside its range, baseline outside.

Fine-tune on our lab traces (E4).

Multi-fault outputs (Zenodo labels).

Time-frequency CNN (Chang & Park).

Better signals

Matched-load reference subtraction (cancels generator ringing).

74AC14 fast-edge pulser: ≈ 2× resolution.

Averaging + narrow pulses for long cables.

Better system

Second cable type (RG-174 / RG-59).

Compact k-NN (shared neighbours) or distillation.

Live capture inside the dashboard.

Confidence-based abstention.

Conclusion

- **Problem:** locate and classify cable faults from one TDR trace, robustly in noise and with real instruments.
- **Solution:** physics simulator → one processing chain → classical baseline vs five ML families on two inputs, calibrated to the setup.
- **Built with:** Python, NumPy/SciPy, scikit-learn, PyTorch, PyVISA, Streamlit/Plotly; tested with 62 unit tests and a 19-feature self-test.
- **Achieved:** Extra Trees (features) locates every fault with MAE 0.37 m (simulation); 1-D CNN (raw) classifies with macro-F1 0.983; tested on 16 real traces.
- **Remaining:** our own lab session: calibration, labelled traces, E3/E4 on real data, final report sections.
- **Potential:** a low-cost, explainable cable diagnostic for student labs and field maintenance.

Thank you: questions?

Demo: python index.py dashboard · Evidence: report/results/

Docs: docs/PROJECT_DOCUMENTATION.md

Technical appendix and source references follow.

Appendix

Technical appendix

Formulas, configuration, file formats and source references (docs/TECHNICAL_APPENDIX.md)

Formulas

Quantity	Formula
Wave speed	vp = c/√εr = VF·c
Incident step	V_inc = Vs·Z0/(Rs + Z0)
Reflection coefficient	Γ = (ZL − Z0)/(ZL + Z0)
Series / shunt R	Γ = R/(R + 2Z0) / Γ = −Z0/(Z0 + 2R)
Distance	d = vp·Δt/2
Calibration	Δt = (2/vp)·L + t0
Resolution	Δd_min ≈ vp·t_r/2
Loss	α(f) = α_dc + k_s√f + k_d·f; Γ ≈ amp·e^(2αd)
Robust noise	σ = 1.4826·median|x − median(x)|
MAE / RMSE	mean|d̂ − d| / √mean(d̂ − d)²
Precision / recall / F1	TP/(TP+FP) · TP/(TP+FN) · 2PR/(P+R)

Configuration values

Setting	Value	Setting	Value
distance	0.5–20 m	rise time	2–10 ns
SNR	10–40 dB	generator Rs	0.92–1.08 × Z0
scope input	1 MΩ ‖ 8–20 pF	sampling	0.5, 1, 2 ns
partial |Γ|	0.05–0.70	ADC	8 bit
Method A grid	256 × 1 ns, −16…239 ns	Method B	23 features
forest	100 trees	k-NN	k ∈ {5, 15}, distance
MLP	(128,64)/(256,128), α 1e-4	CNN	16/32/64/64 ch, k7, ≤ 40 epochs
train / val / test	20 000 / 4 000 / 6 000	seeds	1 / 2 / 3

Source references: where every claim comes from

Topic	Source file(s)
Physics, simulator	src/sim_tdr.py, docs/theory_sheet.md, tests/test_physics_baseline.py
Baseline, features	src/baseline.py, src/features.py, docs/data_dictionary.md
Models, grids, selection	src/models.py, src/train.py, report/results/validation_grid.csv, analysis_model_config.csv
Simulation results E1–E6	report/results/summary.md, e*_*.csv, report/figures/fig1–fig10
Localization / detection analysis	report/results/ml_analysis_summary.md, analysis_*.csv, figures a1–a6
Real data	report/results/zenodo_summary.md, zenodo_z*.csv, figures zfig1–3
Testing	report/results/feature_tests.md, tests/, docs/TESTING_REPORT.md
Hardware	hardware/README.md, HARDWARE_GUIDE.md, LAB_PROCEDURE.md, circuits/
Literature	docs/literature.md, docs/model_selection.md